



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF DATA SCIENCE AND ENGINEERING

**AI-Powered Stock Evaluation Model:
Integrating Technical, Social Sentiment,
Derivative, and Fundamental Signals
for Comprehensive Analysis**

Supervisor:

Md. Easin Arafat

Assistant Lecturer

Author:

Umer Shehzad

Computer Science BSc

Budapest, 2026

Thesis Registration Form

Attention! The deadline for amendment is 1 February in case of final exam in June and 31 August in case of final exam in January.

Student's Data:

Student's Name: Shehzad Umer

Student's Neptun code: CNWF5B

Educational Information:

Training programme: Computer Science BSc

I have an internal supervisor

Internal Supervisor's Name: *Arafat Md Easin*

Supervisor's Home Institution: **Department of Data Science and Engineering**

Address of Supervisor's Home Institution: **1117, Budapest, Pázmány Péter sétány 1/C.**

Supervisor's Position and Degree: *PhD Candidate*

Thesis Title: AI-Powered Stock Evaluation Model: Integrating Technical, Social Sentiment, Derivative, and Fundamental Signals for Comprehensive Analysis

Topic of the Thesis:

(Upon consulting with your supervisor, give a 150-300-word-long synopsis of your planned thesis.)

Problem to be solved:

Traditional stock evaluation methods focus on single factors, such as price trends or fundamental ratios, missing critical insights available from integrating investor social sentiment, technical indicators, options activity, and short interest. These gaps limit the depth and utility of stock assessments for both individuals and institutions.

Motivation:

Most traditional stock analysis relies on a single method, overlooking the complex realities of the market. By bringing together technical signals, investor sentiment, derivatives, and fundamentals in one unified system, this project aims to overcome those limitations—delivering a more complete, realistic, and actionable view for investors seeking deeper insight and better decision-making.

Method of operation:

The system automatically collects stock prices, technical indicators, options activity, short interest, fundamental ratios, and real-time social sentiment from platforms such as Reddit and Twitter, using pre-built APIs (e.g., yfinance, Finnhub Social Sentiment).

Workflow:

User enters a stock ticker → System fetches all relevant data via APIs → Technical, sentiment, derivative, and fundamental scores are calculated using clear rules → Composite score is computed through weighted averaging → The dashboard displays the total score and breakdown for user interaction; no historical data is stored.

Implementation:

A web application is developed using Python for backend data processing, scoring, and integration. The frontend is built with Streamlit for rapid prototyping and interactive display. All calculations are performed in real time per user request. Optional caching for 1-hour may be used to reduce API calls for repeated queries.

Output:

A functional dashboard that delivers instant, multi-signal stock evaluation, detailed technical and user documentation, and a comparative analysis with existing solutions (Prospero.ai startup)—demonstrating the system's accessibility and effectiveness for retail investors.

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Goals	8
1.3	Scope of the Project	9
2	User Documentation	11
2.1	Introduction	11
2.2	Methods and Tools Used	11
2.2.1	Python and Data Science Libraries	11
2.2.2	Groq AI API (LLM Sentiment Engine)	12
2.2.3	Streamlit	12
2.2.4	BeautifulSoup and Requests	12
2.3	User Guide	12
2.3.1	Hardware and Software Requirements	12
2.4	Setting Up the Application Locally	13
2.4.1	Prerequisites	13
2.4.2	Installation Steps	13
2.4.3	Project File Structure	14
2.4.4	Troubleshooting	15
2.5	Features of the Stock Evaluation	15
2.5.1	Ticker / Company Name Entry and Resolution	15
2.5.2	Composite Score and Signal Strength Rating	18
2.5.3	Insider Activity Banner	19
2.5.4	Signal Breakdown: Fundamentals Panel	19
2.5.5	AI Sentiment Panel	20
2.5.6	Technical Analysis Panel	21
2.5.7	Derivatives and Options Panel	22

2.5.8	Interactive Price Chart	23
3	Developer Documentation	24
3.1	Introduction	24
3.2	Problem Specification	24
3.3	System Architecture	25
3.4	Class Diagram	25
3.5	Use Case Diagram	26
3.6	User Sequence Diagram	27
3.7	Tools and Methods Used	28
3.7.1	Data Sources and APIs	28
3.7.2	Python Libraries	29
3.7.3	Rationale for Technology Choices	29
3.8	Core Components	29
3.8.1	<code>data_loader.py</code> : DataLoader Class	29
3.8.2	<code>scorers.py</code> : ScoringEngine Class	31
3.8.3	<code>utils.py</code> : Utility Functions	32
3.8.4	<code>main.py</code> : Streamlit Orchestration	32
3.9	Scoring Algorithms	33
3.9.1	Technical Scoring	33
3.9.2	AI Sentiment Scoring	34
3.9.3	Derivatives Scoring	34
3.9.4	Fundamental Scoring: Five-Pillar Model	36
3.9.5	Composite Score Calculation	37
3.10	Testing	38
3.10.1	Manual Testing Approach	38
3.10.2	Expected Dashboard Output	39
3.10.3	Edge Case Handling	39
3.10.4	Unit Testing	44
4	Conclusion and Future Work	50
4.1	Introduction	50
4.2	Conclusion	50
4.3	Future Work	51

CONTENTS

Acknowledgements	53
Bibliography	53
List of Figures	56
List of Tables	58

This project has been deployed online and can be accessed at:
<https://ai-powered-stock-evaluationthesis-um6.streamlit.app/>

Github link:
<https://github.com/ELTE-DSED/AI-Powered-Stock-Evaluation-Thesis>

Chapter 1

Introduction

Stock markets generate more data than most investors can meaningfully process: price trends, financial statements, news coverage, derivatives activity, and more. Despite this, most tools available to individual investors analyze one signal type at a time. A trader might examine a price chart or review a company's fundamentals, but rarely both — and almost never all relevant signal types together. This fragmented approach leaves gaps and can lead to misleading conclusions.

This thesis describes the development of an AI-powered stock evaluation system that addresses this limitation by integrating four analytical dimensions: technical indicators, AI-based news sentiment, derivatives data, and fundamental analysis. Each dimension captures a different aspect of a stock's condition, and when combined into a single composite score, they produce a more complete picture than any individual method alone.

1.1 Motivation

This project arose from a pattern observed on social media. Monitoring prominent finance professionals on platforms such as X revealed a recurring trend: many of these accounts were endorsing the same application, Prospero AI, with enough consistency to warrant a closer look.

Prospero AI works by assigning a sentiment score to a stock based on recent news articles, then combining that score with fundamental data, technical indicators, and derivatives data in a single dashboard. What was notable was not the sophistication of the underlying algorithms, which were fairly straightforward. What stood out was

that this kind of integrated view did not really exist elsewhere. Most tools available to retail investors focus on one signal category, either a price chart or a fundamental ratio like P/E. Some may combine two, but bringing all four together remained uncommon. Prospero identified this gap and, at the time of writing, had reached a reported valuation of approximately \$16 million.

This points to a real problem. Markets produce vast amounts of data daily, including price movements, earnings reports, news, and options activity. Yet most retail-facing tools process only a small fraction of these signals. That selective view can produce substantive errors. A stock might show a clean upward trend on a price chart while options traders are quietly accumulating put positions, which typically signals an expected decline. Or a stock might look expensive on P/E while showing strong revenue growth and positive news coverage. Looking at one signal in isolation risks missing the full picture.

The motivation for this thesis is straightforward: combining four signal categories produces a more complete and actionable view of any given stock than any single method can. This is the principle behind Prospero AI, and this project builds on that idea in a more structured and transparent way.

The timing also works in this project's favour. Large language models have improved considerably in recent years, and services like Groq now offer fast, low-cost inference. Classifying the sentiment of dozens of news headlines in real time, within the time a web page loads, is now practical. A few years ago, building something like this would have been computationally or financially out of reach for a single developer.

1.2 Goals

- Acquire and process live market data from public APIs, including historical price series, fundamental ratios, options chain details, short interest figures, and insider transaction records.
- Compute a Technical Score from eight market signals, including SMA/EMA crossovers, RSI, MACD crossovers, and Bollinger Band positioning.
- Deploy a Llama 3 language model via the Groq API to classify the sentiment of recent news headlines and assign an impact score to each.

- Derive a Derivatives Score by examining Put/Call ratios, short float percentage, days-to-cover, and implied volatility relative to the prevailing price trend.
- Evaluate the stock across five fundamental domains: Profitability, Growth, Valuation, Financial Health, and Free Cash Flow Quality, supplemented by a Piotroski F-Score.
- Consolidate the four module scores into a weighted Composite Score (0–100), mapped to a qualitative rating scale from *Strong Bearish* to *Strong Bullish*.
- Present all outputs through an interactive Streamlit dashboard incorporating live candlestick charts, signal visualizations, and expandable news sections.

Secondary objectives include the development of comprehensive technical and user documentation to facilitate system understanding and future expandability, alongside conducting a preliminary comparative analysis with selected existing commercial platforms offering analogous functionalities.

1.3 Scope of the Project

This project focuses on combining heterogeneous signal types rather than training predictive models. Scoring logic is rule-based and transparent by design, which makes the system easier to inspect and extend. The implementation covers the following:

- Data acquisition from Yahoo Finance via the `yfinance` library, from FinViz via web scraping, and from the Groq API for language model inference.
- Technical indicator computation using the `ta` (Technical Analysis) Python library.
- A fundamental scoring engine built around five analytical pillars, with P/E ratio benchmarking across twelve industry sectors.
- An AI sentiment pipeline backed by a large language model, with API key pooling and a model fallback mechanism to handle service interruptions.
- A derivatives scoring module that reads options and short-selling data in the context of the current price trend.

- An insider transactions signal derived from buy and sell counts on Yahoo Finance, applied as a marginal adjustment to the composite score.
- A Streamlit dashboard deployable on Streamlit Community Cloud or any Python-compatible environment.

The system is limited to US-listed equities. It is intended as a research and educational prototype, not financial advice, and not a substitute for broader investment analysis.

Chapter 2

User Documentation

2.1 Introduction

This chapter provides user documentation for the AI-Powered Stock Evaluation System. The system is a real-time prototype that evaluates any US-listed stock across four signal types: technical indicators, AI-classified news sentiment, derivatives market data, and fundamental financial ratios. The resulting outputs are presented in a single interactive dashboard. The chapter covers installation requirements, configuration steps, and guidance on interpreting the system's output.

2.2 Methods and Tools Used

The system relies on several Python libraries and external services to perform analysis in real time. The core technologies are described below.

2.2.1 Python and Data Science Libraries

The backend is written entirely in Python. The **Pandas** library handles tabular data structures and data manipulation. The **yfinance** library interfaces with Yahoo Finance to retrieve price history, fundamental ratios, options chains, balance sheet data, and insider transaction records. Technical indicators are computed via the **ta** (Technical Analysis) library, which provides implementations of all indicators used in this project.

2.2.2 Groq AI API (LLM Sentiment Engine)

Real-time sentiment classification is performed using Meta’s Llama 3.3 70B and Llama 3.1 8B models, deployed on Groq’s inference infrastructure. Groq was selected on account of its substantially lower latency relative to comparable inference services. For each analysis request, up to thirty recent news headlines are scraped from FinViz and submitted to the Groq API as a single batch. The model returns a sentiment classification for each headline (Bullish, Bearish, or Neutral) alongside an impact score ranging from 0 to 10. API key pooling and automatic model fallback are implemented to ensure continuity: if one key reaches its rate limit, the system falls back to the next available key without interrupting the analysis.

2.2.3 Streamlit

The user interface is built with Streamlit, a Python framework for developing web applications with minimal frontend configuration. Custom CSS, injected via `st.markdown`, governs the visual presentation, including the dark theme, gradient headers, rounded card components, and colour-coded signal indicators. Interactive candlestick charts with technical indicator overlays are rendered using Plotly.

2.2.4 BeautifulSoup and Requests

FinViz news headlines are retrieved via HTTP requests using the `Requests` library and parsed with `BeautifulSoup`’s HTML parser. The originating publication for each headline, such as Yahoo Finance, Seeking Alpha, or MarketWatch, is extracted from the URL domain.

2.3 User Guide

2.3.1 Hardware and Software Requirements

The application supports both local deployment and access via a hosted Streamlit instance. The minimum and recommended hardware specifications for local deployment are listed in Table 2.1.

Table 2.1: Hardware and Software Requirements

Component	Minimum	Recommended
Processor	2.0 GHz x86- or x64-bit dual-core	3.5 GHz or faster 64-bit quad-core
Memory	4 GB RAM	8 GB RAM or more
Display	Super VGA (1024 x 768 resolution)	Full HD (1920 x 1080 resolution)

2.4 Setting Up the Application Locally

The following steps cover local installation and configuration of the project.

2.4.1 Prerequisites

The following tools must be present before installation:

- **Python 3.9 or higher:** available at python.org
- **pip:** the Python package manager, included with Python 3.9+
- **Git:** required for repository cloning, available at git-scm.com
- **A Groq API key:** obtainable at console.groq.com (a free tier is available)

2.4.2 Installation Steps

Step 1: Clone the repository from GitHub:

```
1 git clone https://github.com/ELTE-DSED/AI-Powered-Stock-Evaluation-  
Thesis
```

Step 2: Navigate into the project directory and install all required dependencies:

```
1 cd AI-Powered-Stock-Evaluation-Thesis  
2 pip install -r requirements.txt
```

Step 3: Create a `.env` file in the root directory and populate it with one or more Groq API keys. Multiple keys may be provided as a comma-separated list to enable key pool rotation:

```
1 GROQ_KEYS=your_groq_api_key_1,your_groq_api_key_2
```

Step 4: Launch the Streamlit application:

```
1 python -m streamlit run main.py
```

The application will open automatically in the default browser at `http://localhost:8501`.

2.4.3 Project File Structure

The repository is organised as a flat, modular Python package, with each file carrying a single, well-defined responsibility.

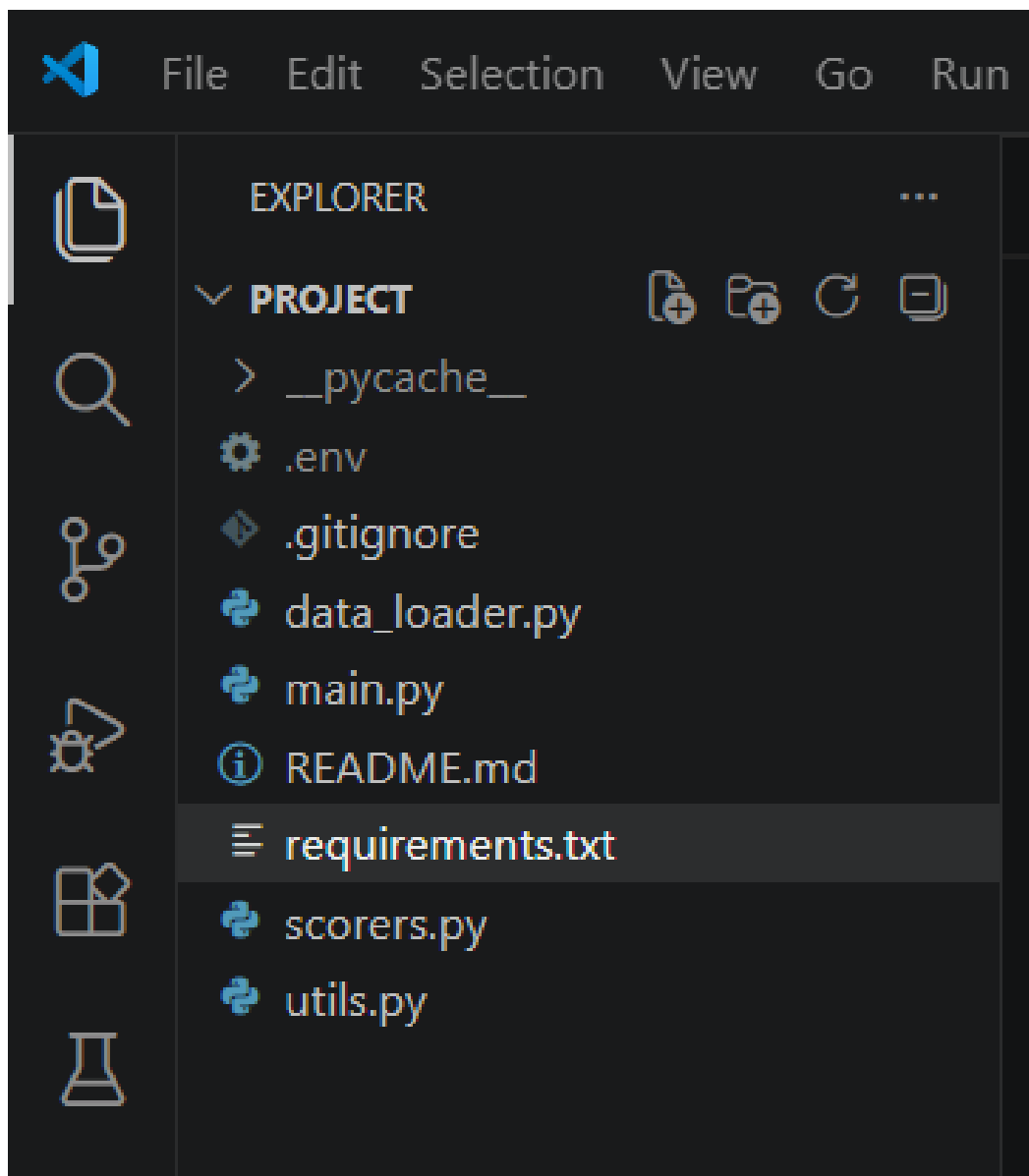


Figure 2.1: Project File Structure in VS Code

2.4.4 Troubleshooting

The following describes common issues and their resolutions:

- **No data returned for ticker:** The symbol may not correspond to a supported US exchange. Entering the full company name (e.g. *Nvidia*) can help resolve ambiguous or alternative ticker formats.
- **Groq API error:** Verify that the `GROQ_KEYS` environment variable is correctly configured. If a key has reached its rate limit, the system will automatically fall over to the next available key in the pool.
- **FinViz blocked / 0 headlines:** FinViz periodically rate-limits scraping requests. Waiting approximately 60 seconds before retrying is generally sufficient to resolve this.
- **ModuleNotFoundError:** Execute `pip install -r requirements.txt` to ensure all required dependencies are installed.

2.5 Features of the Stock Evaluation

2.5.1 Ticker / Company Name Entry and Resolution

The search field accepts either a ticker symbol (e.g. `AAPL`) or a company name (e.g. *Apple*, *Nvidia*, *Berkshire Hathaway*). The `convert_name_to_ticker()` function in `data_loader.py` resolves company names to their corresponding ticker symbols by querying the Yahoo Finance v1 search API and filtering results to NYSE, NASDAQ, and AMEX exchanges. This removes the requirement for users to know the exact ticker symbol in advance.

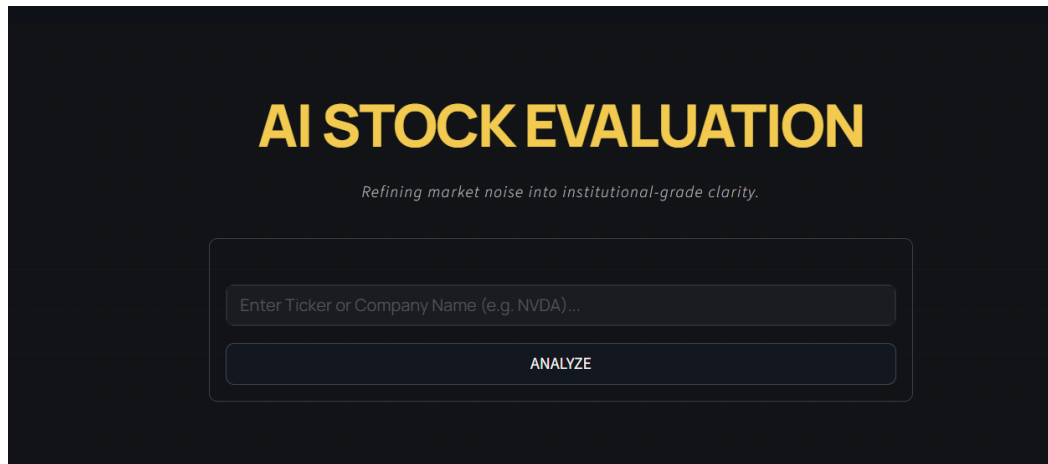


Figure 2.2: Main Dashboard - Ticker Search Input

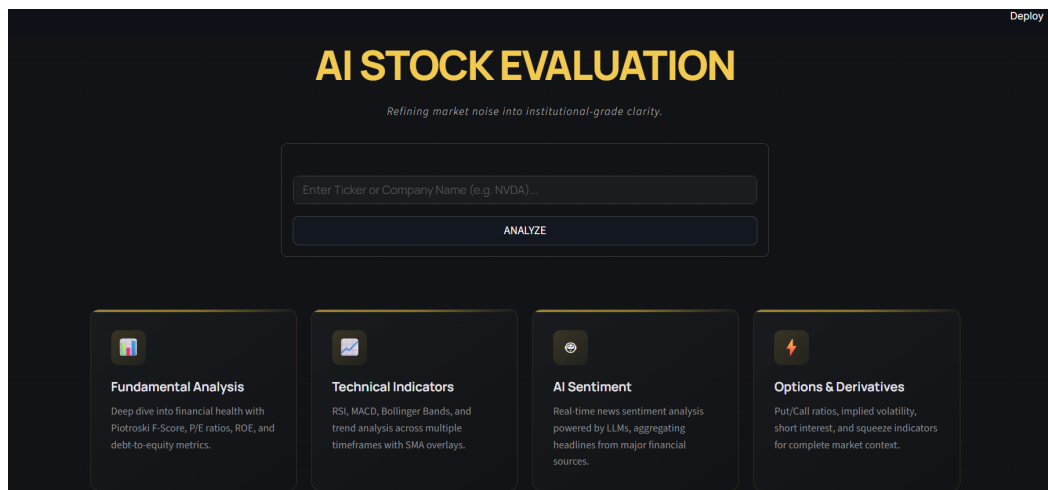


Figure 2.3: Main Dashboard - Landing Page Overview

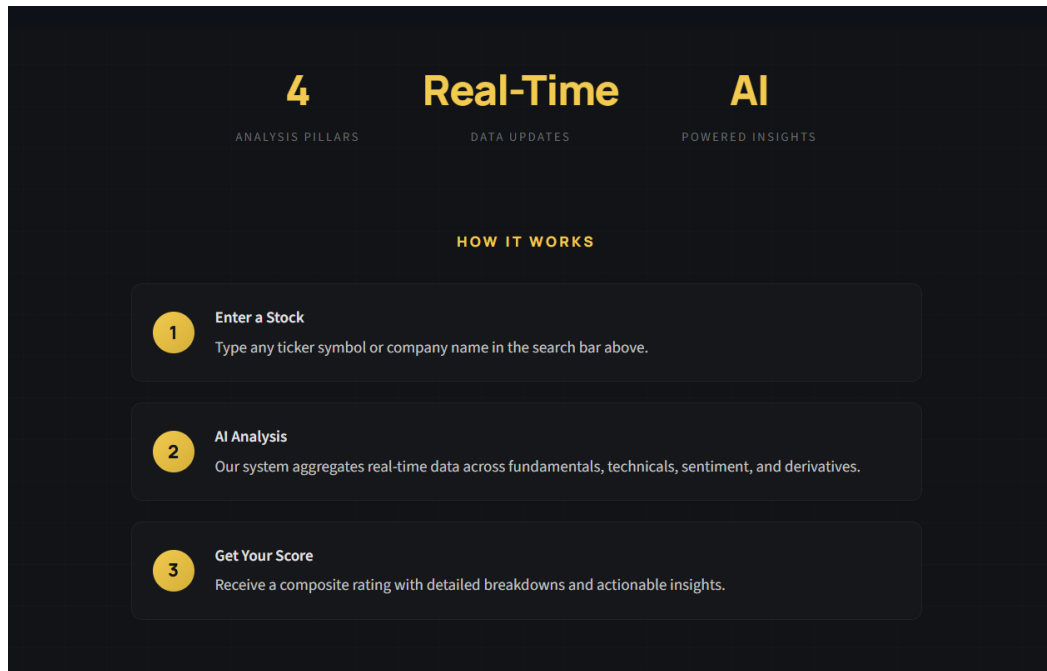


Figure 2.4: Main Dashboard - Workflow Explanation

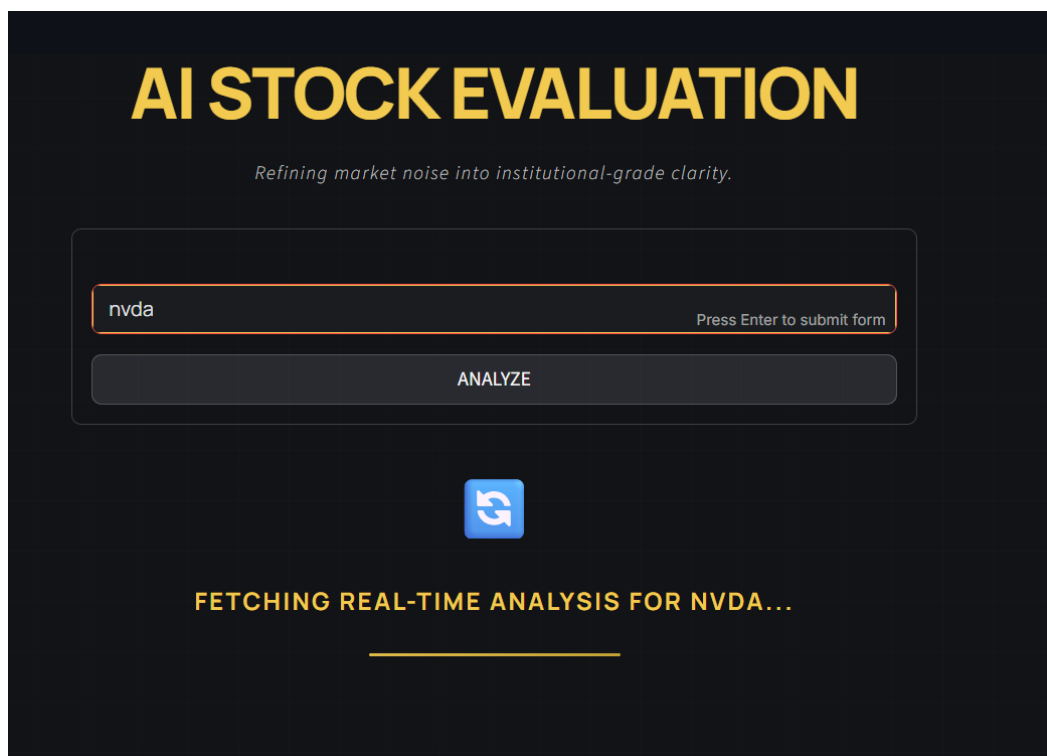


Figure 2.5: Main Dashboard - Fetching Screen During Analysis

2.5.2 Composite Score and Signal Strength Rating

Upon completion of the analysis, the dashboard header displays three metric cards: the Composite Score (0–100), the Signal Strength label (e.g. *Strong Bullish*), and the current closing price. The Composite Score is derived by combining the four module scores using fixed weights. A non-linear stretching function is applied to scores above 72 or below 32, which increases separation between clearly bullish and clearly bearish readings and reduces score clustering around the neutral midpoint.

The five rating bands are defined in Table 2.2:

Table 2.2: Composite Score Rating Scale

Score Range	Rating Label	Interpretation
80 - 100	Strong Bullish	Multiple highly positive signals across all modules
60 - 79	Bullish Bias	Mostly positive signals, some minor concerns
40 - 59	Neutral / Mixed	Conflicting signals, no clear direction
20 - 39	Bearish Bias	Mostly negative signals detected
0 - 19	Strong Bearish	Weakness across all evaluation pillars

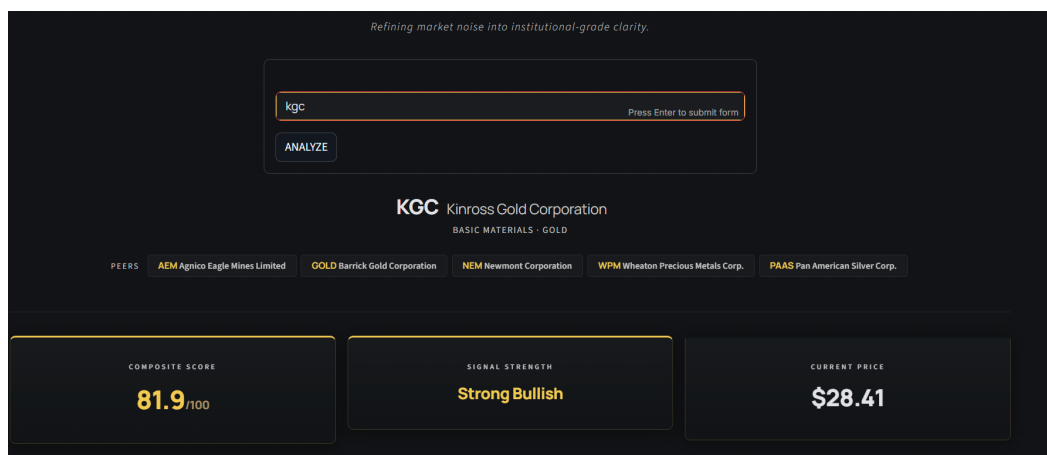


Figure 2.6: Top Metric Cards - Composite Score, Signal Strength, and Current Price

2.5.3 Insider Activity Banner

Directly beneath the metric cards, a colour-coded banner displays insider buying and selling activity recorded over the preceding twelve months. More than ten distinct buy transactions triggers the label *Massive Insider Cluster Buying*; between one and ten buy transactions produce *Minor Insider Buying Detected*. Sell activity does not generate a labelled alert, though the raw sell count remains visible. Links to OpenInsider and the Yahoo Finance insider transactions page are provided for independent verification.

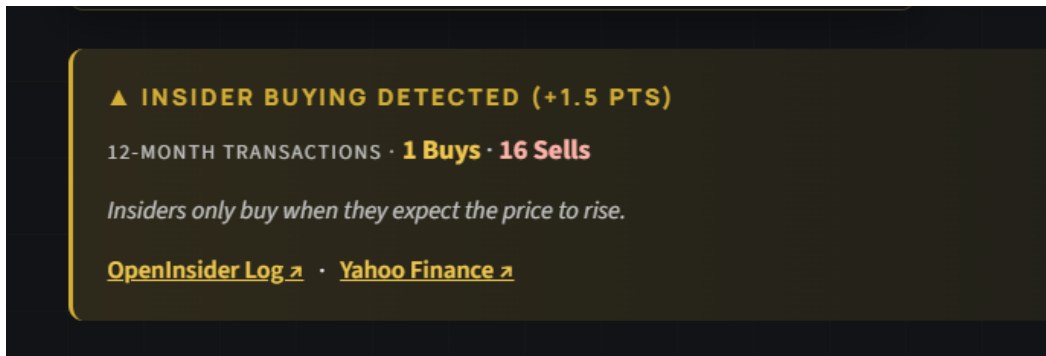


Figure 2.7: Insider Activity Banner - showing buy/sell counts.

2.5.4 Signal Breakdown: Fundamentals Panel

The left column of the Signal Breakdown section contains several vertically arranged analysis panels. The Fundamentals panel presents the overall fundamental score as a progress bar (0–100), followed by a five-pillar breakdown rendered as horizontal mini-bars. Below this, six key metrics are displayed in a two-column grid: P/E Ratio (with the corresponding sector benchmark), P/B Ratio, Return on Equity, Net Margin, Revenue Growth, and Debt-to-Equity Ratio.

An expandable *Piotroski F-Score Signals* section presents pass/fail indicators for each of the nine Piotroski criteria, providing a secondary quality assessment. A link to the Yahoo Finance financial statements page is included for further reference.

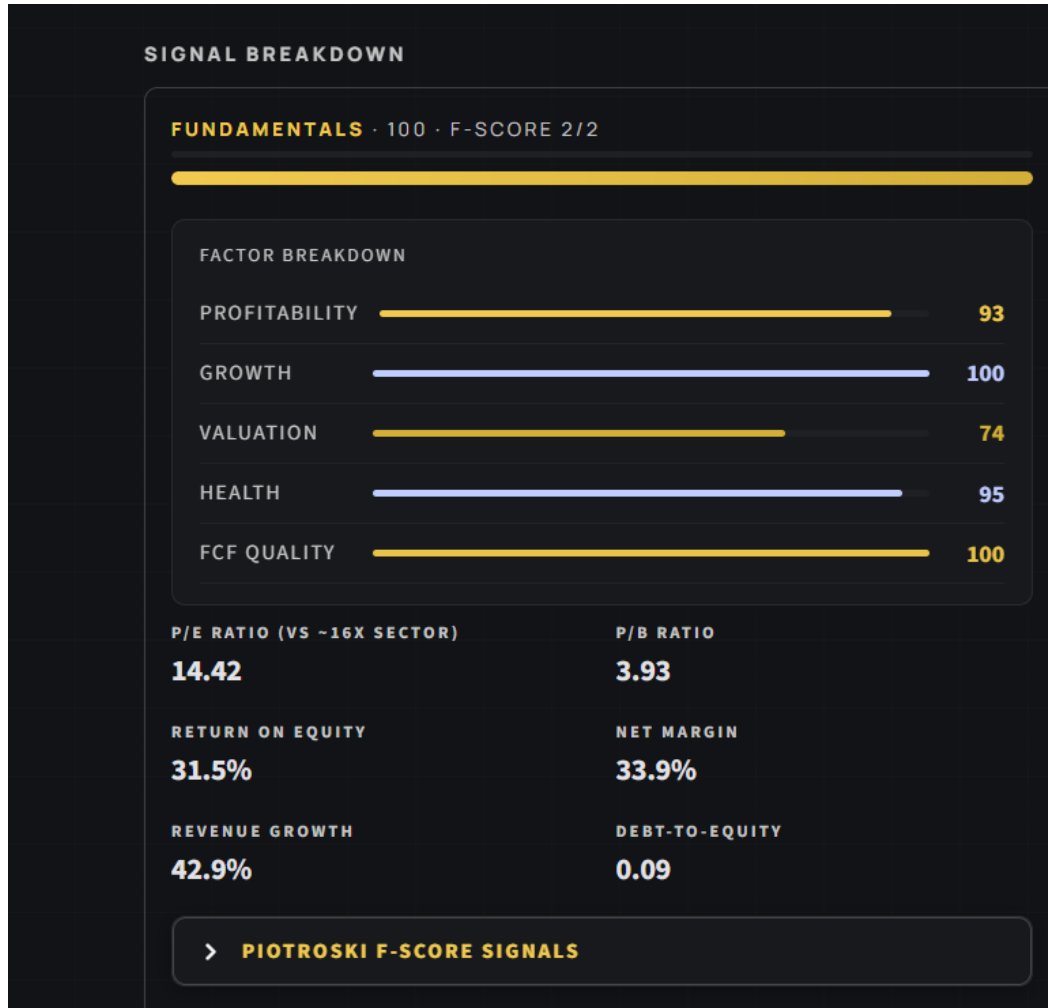


Figure 2.8: Fundamentals Panel

2.5.5 AI Sentiment Panel

The AI Sentiment panel presents the sentiment score as a progress bar, accompanied by a summary line indicating the dominant classification and signal count (e.g. *Bullish Bias: 12 positive signals*). Three count indicators display the number of headlines classified as Bullish, Neutral, and Bearish by the language model. An expandable *View Analyzed Headlines & Sources* section lists each headline with its source publication, timestamp, sentiment classification, impact score, and a direct link to the original article. Headlines are ordered by descending impact score, placing the most analytically relevant items first.

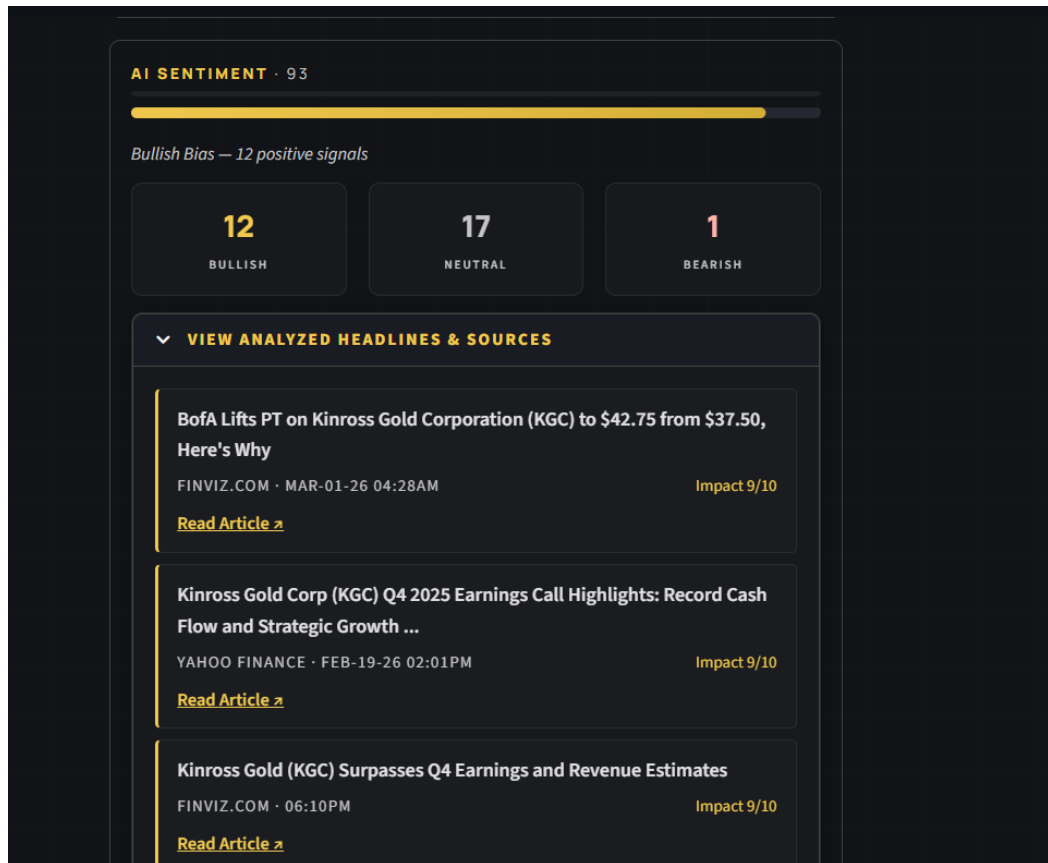


Figure 2.9: AI Sentiment Panel

2.5.6 Technical Analysis Panel

The Technical Analysis panel displays RSI (14-period) and MACD status in the left column, with trend classification relative to the 200-day SMA and Bollinger Band positioning in the right column. Colour-coded signal indicators below these fields present the pass/fail status of each of the eight technical signals. A reference link to TradingView is provided for further charting analysis.

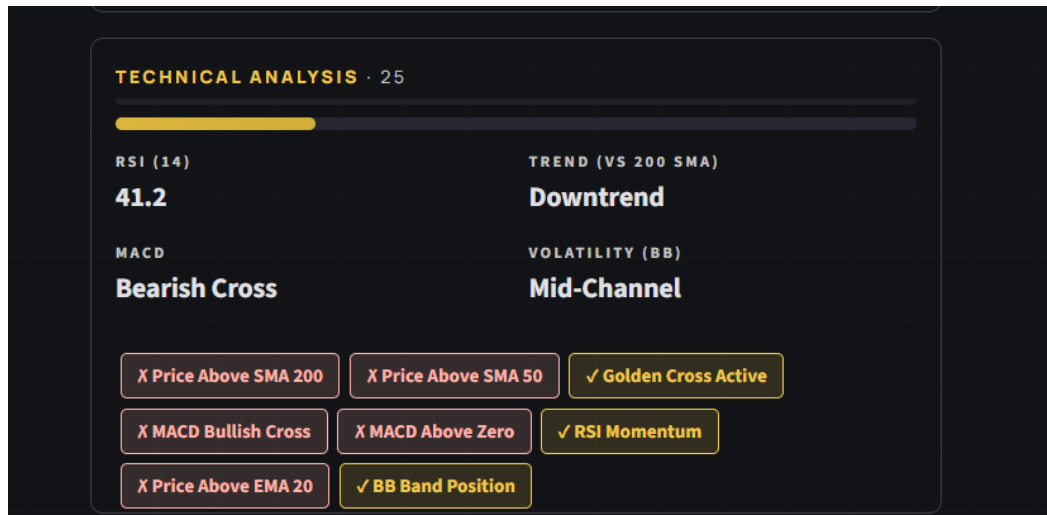


Figure 2.10: Technical Panel

2.5.7 Derivatives and Options Panel

The Derivatives panel reports the Volume Put/Call Ratio, Open Interest Put/Call Ratio, Short Float percentage, Days to Cover (short ratio), and average Implied Volatility. A *Squeeze Watch* alert is triggered when the short ratio exceeds eight days to cover and the short float exceeds ten percent in an uptrending stock, a combination historically associated with conditions conducive to short-squeeze events.

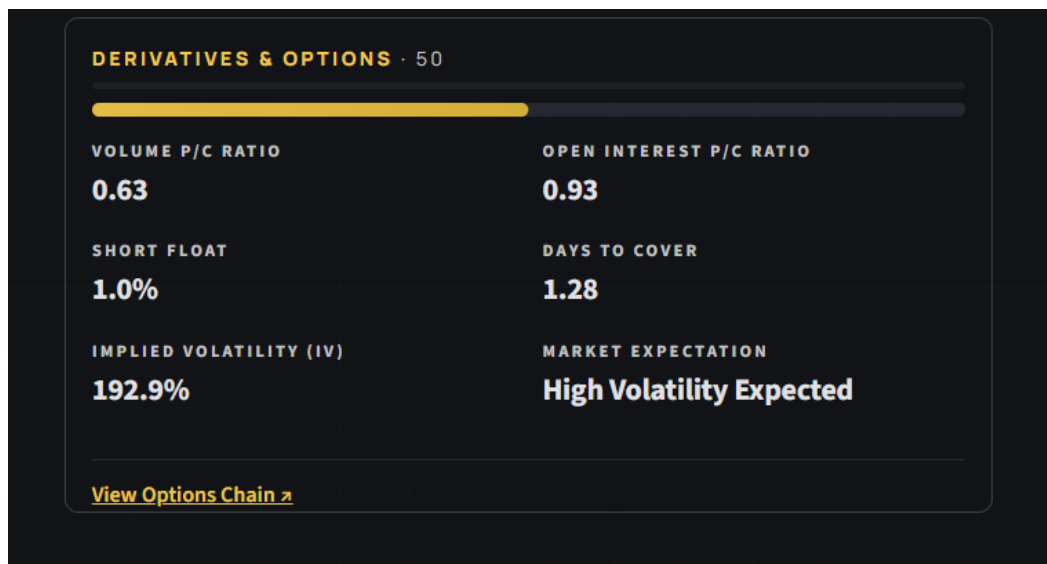


Figure 2.11: Derivatives Panel

2.5.8 Interactive Price Chart

The right column of the Signal Breakdown section contains an interactive Plotly candlestick chart covering the preceding twelve months of price data. The chart overlays the 50-day SMA (blue), the 200-day SMA (red dashed line), and Bollinger Bands (grey shaded region). Zoom, pan, and hover interactions are supported, allowing inspection of exact OHLC values for any individual trading session.



Figure 2.12: Interactive Candlestick Chart

Chapter 3

Developer Documentation

3.1 Introduction

This chapter covers the internal architecture of the AI-Powered Stock Evaluation System: its module structure, external dependencies, scoring logic, testing strategy, and potential extension points.

3.2 Problem Specification

The core problem this system addresses is that no single analytical method is sufficient to evaluate a stock reliably. Relying exclusively on any single source — whether price charts, financial statements, news coverage, or options data — produces an incomplete picture. This system treats each signal type as an independent scoring module, normalises all outputs to a common 0–100 scale, and combines them through a research-informed weighting scheme.

Three design principles shaped the architecture. First, all data must be fetched in real time at query execution, with no reliance on cached data beyond a one-hour window. Second, each scoring module must operate independently: if a data source is unavailable, that module returns a zero score without affecting the rest of the pipeline. Third, all scoring logic must be rule-based and fully transparent rather than predictive, as interpretability is a core requirement in an academic context.

3.3 System Architecture

The system consists of four Python modules coordinated by a Streamlit presentation layer. The architecture is illustrated in Figure 3.1.

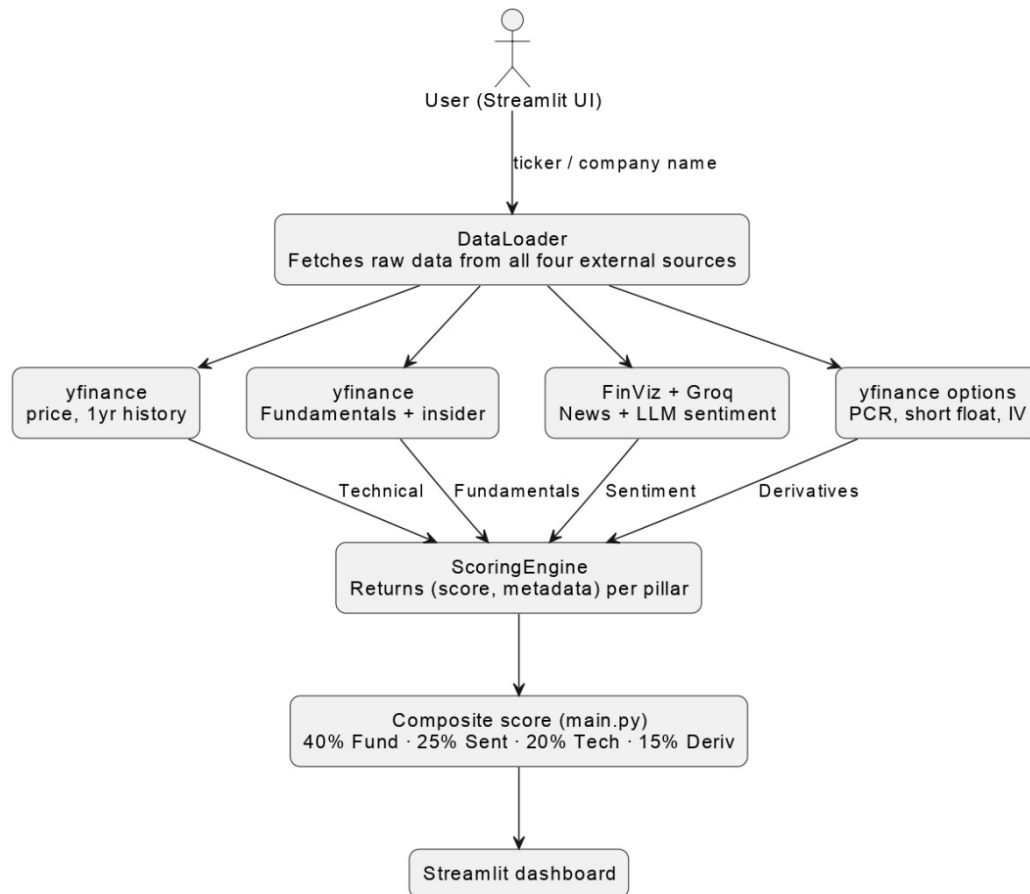


Figure 3.1: System Architecture Diagram

Data flows through the system in a pipeline. The user submits a ticker symbol via the Streamlit interface, after which the `DataLoader` class retrieves raw data from all four external sources. The `ScoringEngine` class then processes each data type and returns a normalised score alongside a metadata dictionary. The four scores are subsequently combined in `main.py` using the composite weighting formula, and the resulting score and metadata are rendered into the dashboard.

3.4 Class Diagram

The class diagram in Figure 3.2 illustrates the structure of the four main system components and their relationships. The `DataLoader` class handles all raw data re-

trieval, exposing separate methods for technical, fundamental, derivative, and social data types. The **ScoringEngine** class processes this raw data into normalised scores and maintains a shared attribute, `current_tech_trend`, which is set during technical scoring and subsequently read by the derivatives scoring method. The **utils** module provides two helper functions available to all components. All modules are coordinated through `main.py`, which serves as the system entry point.

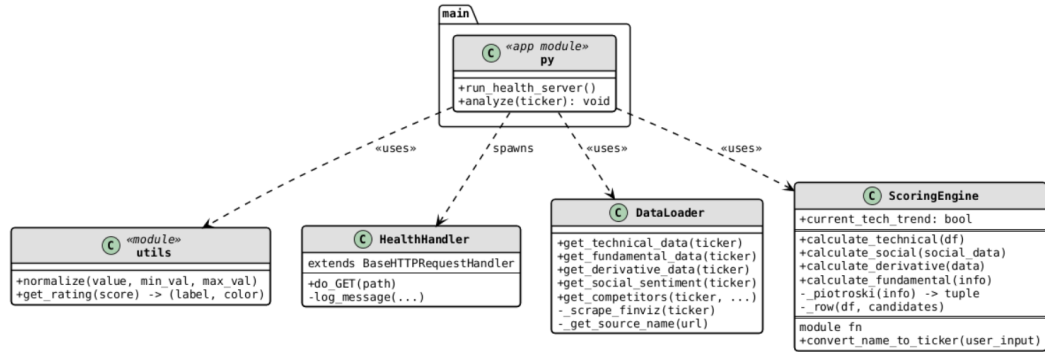


Figure 3.2: Class Diagram

3.5 Use Case Diagram

The use case diagram maps the actions available to the user against the external services invoked behind the scenes. From the user's perspective, there are seven primary interactions, including stock search and access to individual analysis panels. Some use cases depend on more than one external service. Viewing the sentiment panel, for instance, first scrapes headlines from FinViz and then forwards them to the Groq API for classification. The competitor lookup also calls the Groq API independently. Yahoo Finance is the most heavily used external source, supplying data for the technical, fundamental, and derivatives panels simultaneously.

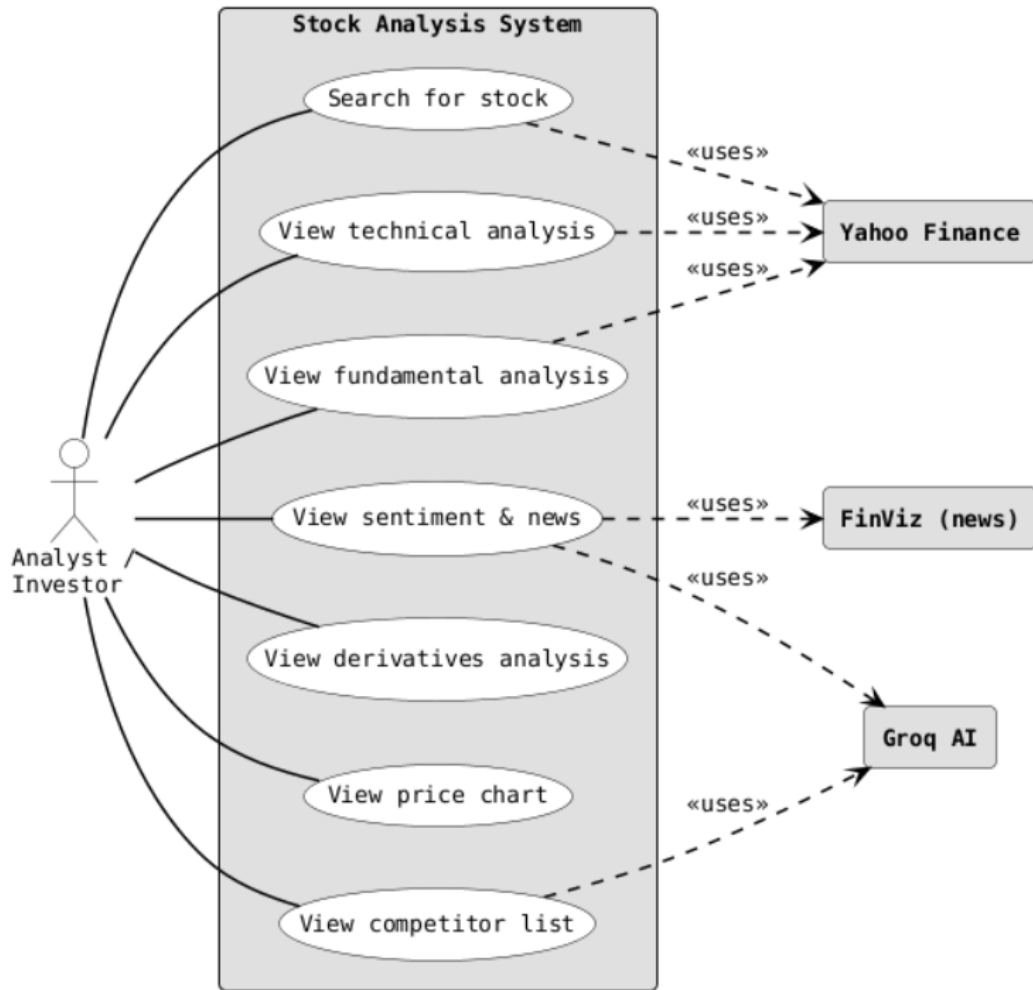


Figure 3.3: Use Case Diagram

3.6 User Sequence Diagram

The sequence diagram illustrates the full lifecycle of a single user request. Upon ticker or company name submission, a name-to-ticker conversion query is dispatched to Yahoo Finance to validate the symbol. A sequential data-fetching loop then executes, during which `DataLoader` retrieves all four data types from their respective external APIs. Once collection is complete, `ScoringEngine` runs four independent scoring calculations, each returning a score and a metadata dictionary to `main.py`. The weighting formula is then applied to the four scores and the completed dashboard is rendered.

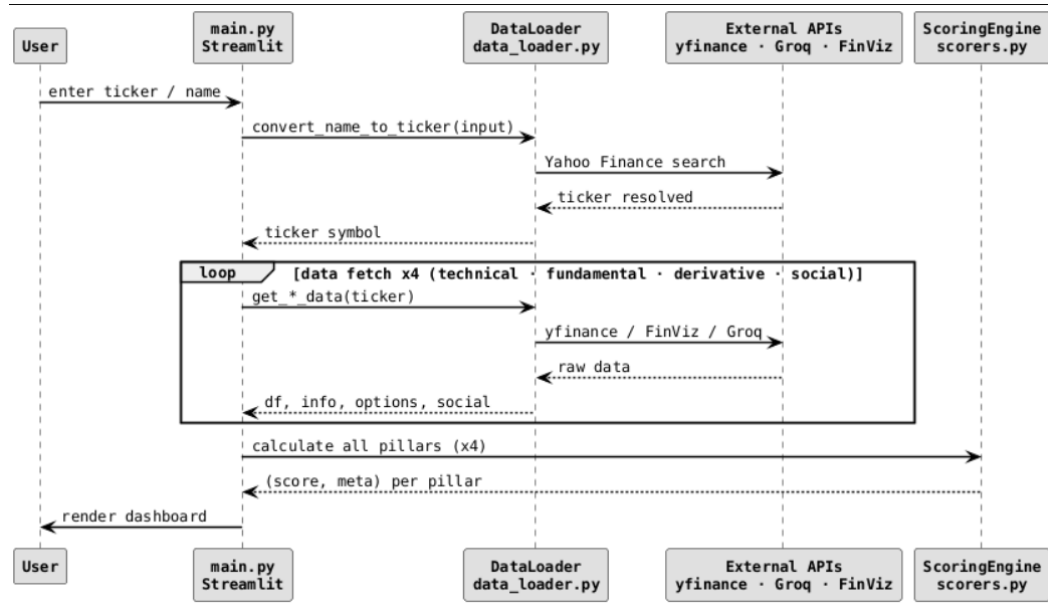


Figure 3.4: User Sequence Diagram

3.7 Tools and Methods Used

3.7.1 Data Sources and APIs

Yahoo Finance (via yfinance):

The `yfinance` library provides access to OHLCV price history, fundamental financial ratios, options chains, balance sheet and cashflow data, and insider transaction records. It functions as a wrapper around Yahoo Finance’s unofficial JSON API.

FinViz:

FinViz is a financial news aggregator from which up to thirty recent headlines are retrieved via HTTP requests and parsed with BeautifulSoup. It was selected because it consolidates US stock news from over twenty sources, including the Wall Street Journal, Bloomberg, Barron’s, MarketWatch, and Seeking Alpha.

Groq API:

The Groq API provides low-latency inference for Meta’s Llama 3.3 70B and Llama 3.1 8B models, used here for batch sentiment classification of scraped headlines. Groq was selected over alternatives such as OpenAI on the basis of inference speed (approximately 500 tokens per second compared to roughly 50 for OpenAI’s GPT-3.5), a meaningful difference when the full analysis must complete within ten seconds.

3.7.2 Python Libraries

The primary Python dependencies are as follows:

- **yfinance (0.2.x)**: Yahoo Finance data wrapper
- **ta (0.11.x)**: Technical Analysis library providing RSI, SMA, EMA, MACD, and Bollinger Band implementations
- **pandas / numpy**: data manipulation and numerical computation
- **requests / beautifulsoup4**: HTTP retrieval and HTML parsing for FinViz
- **groq**: official Groq Python SDK for LLM inference
- **streamlit**: web application framework for the dashboard
- **plotly**: interactive charting library for candlestick and indicator visualisations
- **python-dotenv**: environment variable management for local development

3.7.3 Rationale for Technology Choices

Python was selected as the implementation language given its established role in financial data science and given the availability of domain-specific libraries such as **yfinance** and **ta**. Streamlit was preferred over Flask or Dash because it supports rapid construction of interactive dashboards with minimal frontend configuration, appropriate for a research prototype where iteration speed takes priority over UI customisation. For the LLM sentiment pipeline, Groq was chosen over OpenAI's API primarily on inference speed grounds: at roughly 500 tokens per second, Groq processes the full headline batch within the ten-second response target that OpenAI's GPT-3.5 could not reliably meet.

3.8 Core Components

3.8.1 `data_loader.py`: **DataLoader** Class

The **DataLoader** class handles all data retrieval and exposes four public methods, one per signal type. Each method is designed to fail gracefully: all network calls are

wrapped in `try/except` blocks and return `None` or an empty result on failure, rather than raising an exception that would halt the entire analysis pipeline.

```

53
54 class DataLoader:
55     def __init__(self):
56         pass
57
58 > def get_technical_data(self, ticker): ...
65
66 > def get_fundamental_data(self, ticker): ...
99
100 > def get_derivative_data(self, ticker): ...
138
139 > def _get_source_name(self, url): ...
155
156 > def _scrape_finviz(self, ticker): ...
182
183 def get_social_sentiment(self, ticker):
184     if not API_KEY_POOL:
185         return {"error": "API Keys are missing! Add them to Streamlit Secrets."}, "Error"
186
187     raw_news = self._scrape_finviz(ticker)
188     if not raw_news:
189         return {"error": "FinViz returned 0 articles. It might be a bad ticker or a temporary block."}, "No Data"
190
191     titles_only = [h['title'] for h in raw_news]

```

Figure 3.5: `data_loader.py` — `DataLoader` class with `get_technical_data()`, `get_fundamental_data()`, `get_derivative_data()`, `get_social_sentiment()` methods

`get_technical_data(ticker):`

Retrieves twelve months of daily OHLCV price history via `yfinance`. Returns a Pandas DataFrame indexed by date, or `None` on failure.

`get_fundamental_data(ticker):`

Retrieves the full `info` dictionary from `yfinance` along with insider transaction buy and sell counts. Multi-year financial statement DataFrames required for the Piotroski F-Score calculation are also attached.

`get_derivative_data(ticker):`

Retrieves options chain data for the nearest expiry date falling between 20 and 40 days out, or the closest available expiry if none falls within that range. Put/Call ratios by volume and open interest, average implied volatility, and short interest data from the `info` dictionary are computed and returned.

`get_social_sentiment(ticker):`

Scrapes up to thirty headlines from FinViz, submits them as a batch to the Groq LLM API, and returns structured sentiment classifications and impact scores for each headline. API key pool rotation with model fallback (70B to 8B) is implemented to maintain pipeline continuity.

```

29 # --- SMART SEARCH HELPER ---
30 def convert_name_to_ticker(user_input):
31     clean_input = user_input.strip()
32     if len(clean_input) <= 5 and clean_input.isalpha() and clean_input.isupper():
33         return clean_input
34
35     search_queries = [clean_input]
36     if " " in clean_input: search_queries.append(clean_input.replace(" ", ""))
37     if " and " in clean_input.lower(): search_queries.append(clean_input.lower().replace(" and ", " & "))
38
39     us_exchanges = ['NYSE', 'NASDAQ', 'AMEX', 'OTC', 'ASE', 'PCX']
40     headers = {'User-Agent': 'Mozilla/5.0'}
41
42     for query in search_queries:
43         try:
44             url = f"https://query2.finance.yahoo.com/v1/finance/search?q={query}"
45             response = requests.get(url, headers=headers, timeout=3)
46             data = response.json()
47             if 'quotes' in data and data['quotes']:
48                 for quote in data['quotes']:
49                     if quote.get('quoteType') == 'EQUITY' and quote.get('exchange') in us_exchanges:
50                         return quote['symbol']
51         except: continue
52     return clean_input.upper()

```

Figure 3.6: data_loader.py — convert_name_to_ticker() function showing Yahoo Finance search API call with US exchange filtering

3.8.2 scorers.py: ScoringEngine Class

The `ScoringEngine` class implements all four scoring algorithms as instance methods. A single instance is instantiated per user request in `main.py`. Each method returns a tuple of `(score, meta)`, where `score` is normalised to 0–100 and `meta` contains the intermediate values required for dashboard rendering.

```

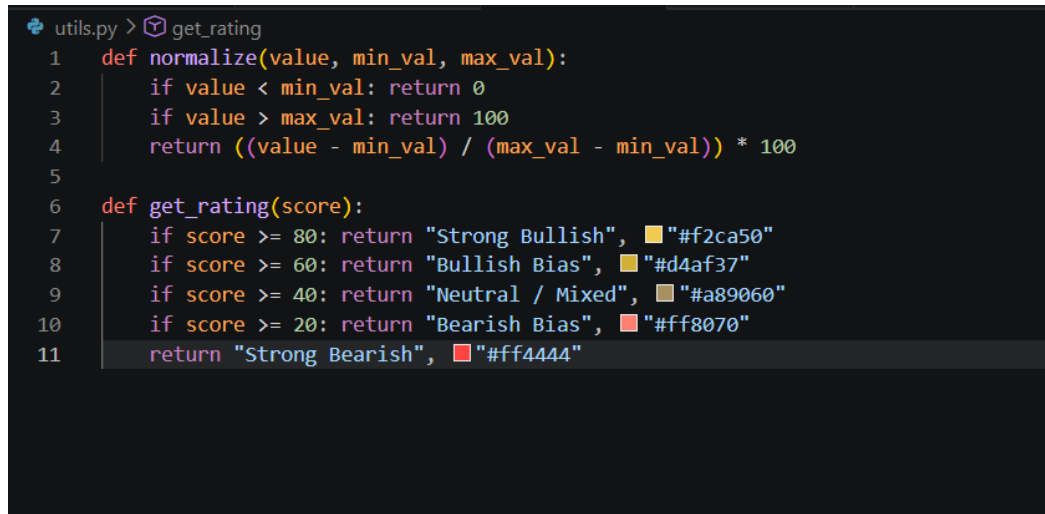
19 #
20 # TECHNICAL - Weighted binary signal count
21 #
22 def calculate_technical(self, df):
23     if df.empty or len(df) < 50:
24         return 0, {}
25
26     close = df['Close']
27     price = close.iloc[-1]
28
29     rsi = RSIIndicator(close).rsi().iloc[-1]
30     sma_50 = SMAIndicator(close, window=50).sma_indicator().iloc[-1]
31     ema_20 = EMAIndicator(close, window=20).ema_indicator().iloc[-1]
32     sma_200 = SMAIndicator(close, window=min(200, len(df))).sma_indicator().iloc[-1]
33
34     macd_calc = MACD(close)
35     macd_line = macd_calc.macd().iloc[-1]
36     macd_signal = macd_calc.macd_signal().iloc[-1]
37
38     bb = BollingerBands(close)
39     bb_high = bb.bollinger_hband().iloc[-1]
40     bb_low = bb.bollinger_lband().iloc[-1]

```

Figure 3.7: scorers.py: ScoringEngine class header and calculate_technical() method showing signal weight definitions

3.8.3 utils.py: Utility Functions

The `utils.py` module provides two utility functions used across the application:



```
utils.py > get_rating
1  def normalize(value, min_val, max_val):
2      if value < min_val: return 0
3      if value > max_val: return 100
4      return ((value - min_val) / (max_val - min_val)) * 100
5
6  def get_rating(score):
7      if score >= 80: return "Strong Bullish", "#f2ca50"
8      if score >= 60: return "Bullish Bias", "#d4af37"
9      if score >= 40: return "Neutral / Mixed", "#a89060"
10     if score >= 20: return "Bearish Bias", "#ff8070"
11     return "Strong Bearish", "#ff4444"
```

Figure 3.8: `utils.py` : `normalize()` and `get_rating()` functions

`normalize(value, min_val, max_val):`

Clamps a value to the specified min/max range and maps it linearly to a 0–100 scale. This function is applied within each scoring method to convert raw metric values to a common scale prior to weighting.

`get_rating(score):`

Accepts a composite score and returns a corresponding label and hex colour string for the UI. A score of 80 or above, for instance, resolves to ('Strong Bullish', '#00ff00').

3.8.4 main.py: Streamlit Orchestration

`main.py` is the application entry point and presentation layer. It configures the Streamlit page, injects custom CSS for the dark theme, renders the search interface, invokes the `DataLoader` and `ScoringEngine` methods, applies the composite weighting and score-stretching logic, and renders all dashboard components.


```
# Weights: Fundamentals 40% | Sentiment 25% | Technical 20% | Derivatives 15%
base_composite = (
    score_fund * 0.40 +
    score_social * 0.25 +
    score_tech * 0.20 +
    score_deriv * 0.15
)
```

Figure 3.9: `main.py`: composite score weighting formula and score stretching logic

3.9 Scoring Algorithms

3.9.1 Technical Scoring

The technical scoring algorithm evaluates eight binary signals, each assigned a weight. A signal contributes $+1$ (bullish) when its condition is satisfied, or -1 (bearish) when it is not. The final technical score is the ratio of total bullish weight to total signal weight, scaled to 100. Signals carrying higher weights (such as price position relative to the 200-day SMA) therefore exert proportionally greater influence on the outcome.

The eight signals and their respective weights are defined in Table 3.1:

Table 3.1: Technical Scoring Signals and Weights

Signal Name	Scoring Logic	Weight
Price Above SMA 200	Bullish if Close > 200-day SMA	25
Price Above SMA 50	Bullish if Close > 50-day SMA	20
Golden Cross Active	Bullish if SMA 50 > SMA 200	15
MACD Bullish Cross	Bullish if MACD Line > Signal	15
MACD Above Zero	Bullish if MACD Line > 0	10
RSI Momentum	Bullish: uptrend with RSI 50–82 or downtrend RSI < 45	8
Price Above EMA 20	Bullish if Close > 20-day EMA	5
BB Band Position	Bullish: uptrend near upper band	2

A notable design decision is the trend-aware RSI rule. An RSI above 50 is treated as bullish only when the stock is in an uptrend (Close > SMA 50), while an RSI below

45 registers as a bearish signal only in a confirmed downtrend. This conditioning prevents an elevated RSI from producing a false bullish reading during a declining price trend, and equally prevents a depressed RSI from registering as bearish during a temporary pullback within an uptrend.

```

22 def calculate_technical(self, df):
23     bb = BollingerBands(close)
24     bb_high = bb.bollinger_hband().iloc[-1]
25     bb_low = bb.bollinger_lband().iloc[-1]
26
27     is_uptrend = price > sma_50
28     self.current_tech_trend = is_uptrend
29
30     # (label, bullish=1/bearish=-1, weight)
31     signals = [
32         ('Price Above SMA 200', 1 if price > sma_200 else -1, 25),
33         ('Price Above SMA 50', 1 if price > sma_50 else -1, 20),
34         ('Golden Cross Active', 1 if sma_50 > sma_200 else -1, 15),
35         ('MACD Bullish Cross', 1 if macd_line > macd_signal else -1, 15),
36         ('MACD Above Zero', 1 if macd_line > 0 else -1, 10),
37         ('RSI Momentum', 1 if (is_uptrend and 50 <= rsi <= 82) or (not is_uptrend and rsi < 45) else -1, 8),
38         ('Price Above EMA 20', 1 if price > ema_20 else -1, 5),
39         ('BB Band Position', 1 if (is_uptrend and price >= bb_high * 0.98) or (not is_uptrend and price < bb_low) else -1,
40
41     ]
42
43     total_w = sum(w for _, w in signals)
44     bull_w = sum(w for _, s, w in signals if s == 1)
45     final_score = (bull_w / total_w) * 100
46

```

Figure 3.10: scorers.py : calculate_technical() method body with signal list and weighted scoring computation

3.9.2 AI Sentiment Scoring

Sentiment scoring is computed from the impact-weighted classifications returned by the Groq LLM. For each headline classified as Bullish, its impact score (0–10) is added to a **bull_power** accumulator; Bearish headlines contribute equivalently to a **bear_power** accumulator. Neutral headlines contribute a small baseline value ($0.2 \times \text{count}$) to a **neutral_anchor**, which dampens score extremity when sentiment is uniformly one-directional across all significant headlines.

The final sentiment score is computed as:

```

1 sentiment_score = ((bull_power + neutral_anchor / 2)
2                  / total_power) * 100

```

This constitutes a power-weighted directional ratio. A score of 50 reflects balanced or negligible net sentiment; scores above 60 indicate a Bullish Bias, while scores below 40 indicate a Bearish Bias.

3.9.3 Derivatives Scoring

The derivatives scoring module incorporates trend awareness, interpreting each metric differently depending on whether the stock is in an uptrend (Close > SMA

```

73 def calculate_social(self, social_data):
74
75     bull_pow = bear_pow = bull_cnt = bear_cnt = neut_cnt = 0
76     for item in headlines:
77         sentiment = item.get('sentiment', '').lower()
78         s = item.get('score', 0)
79         if "bullish" in sentiment: bull_pow += s; bull_cnt += 1
80         elif "bearish" in sentiment: bear_pow += s; bear_cnt += 1
81         else: neut_cnt += 1
82
83     neutral_anchor = neut_cnt * 0.2
84     total_power = bull_pow + bear_pow + neutral_anchor
85     final_score = 50 if total_power == 0 else ((bull_pow + neutral_anchor / 2) / total_power) * 100
86
87     if final_score > 60: summary = f"Bullish Bias - {bull_cnt} positive signals"
88     elif final_score < 40: summary = f"Bearish Bias - {bear_cnt} negative signals"
89     else: summary = "Mixed / Neutral Sentiment"
90
91     return final_score, {
92         "summary": summary, "details": headlines,
93         "counts": {"bull": bull_cnt, "bear": bear_cnt, "neut": neut_cnt}
94     }
95
96
97
98
99
100
101
102
103

```

Figure 3.11: scorers.py: calculate_social() method showing sentiment accumulation and power-weighted scoring formula

50) or a downtrend. This context-sensitivity is a deliberate design choice: the same derivative signal carries different implications under different price regimes.

A high short float (above 10%), for instance, scores at 85/100 in an uptrending stock (where it may indicate short-squeeze conditions) but scores at 15/100 in a downtrending stock, where it instead corroborates bearish momentum. Without trend conditioning, the derivatives score would be directionally symmetric and stripped of the contextual information that makes it analytically useful.

The five derivatives sub-scores and their weights are defined in Table 3.2:

Table 3.2: Derivatives Scoring Sub-Scores and Weights

Signal Name	Scoring Logic	Weight
Short Float %	Uptrend: bullish if < 3%; bearish if > 15%. Downtrend: inverted.	20%
Short Ratio (DTC)	Uptrend: bullish if > 8 DTC + high float (squeeze). Downtrend: inverted.	20%
P/C Ratio (OI)	Bullish if < 0.6 (call-heavy open interest)	30%
P/C Ratio (Vol)	Bullish if < 0.7 (call-heavy volume)	15%
Avg Implied Vol.	Uptrend: lower IV is more bullish. Downtrend: high IV penalised.	15%

```

107     def calculate_derivative(self, data):
108         if not data.get('valid'):
109             return 0, {}
110
111         pcr_vol      = data.get('pcr_vol')
112         pcr_oi       = data.get('pcr_oi')
113         short_float  = data.get('short_float')
114         short_ratio  = data.get('short_ratio')
115         avg_iv       = data.get('avg_iv')
116
117         scores = {}
118
119         if short_float is not None:
120             sf = short_float * 100
121             if not self.current_tech_trend:
122                 float_score = 15 if sf > 10 else 50
123             else:
124                 float_score = 85 if sf < 3 else 30 if sf > 15 else 55
125             scores['float'] = (float_score, 0.20)
126

```

Figure 3.12: scorers.py : calculate_derivative() method showing trend-aware short float scoring branch

3.9.4 Fundamental Scoring: Five-Pillar Model

The fundamental scoring engine applies a five-pillar framework modelled on the structure of professional equity research systems, such as Seeking Alpha’s Quant Ratings. Each pillar is scored from 0 to 100 using threshold-based logic applied to fundamental ratios retrieved from Yahoo Finance. The pillars and their weights are defined in Table 3.3.

Table 3.3: Fundamental Scoring Five-Pillar Model

Pillar	Factors Evaluated	Weight
Profitability	ROE (45%), Net Margin (35%), ROA (20%)	35%
Growth	Revenue Growth (65%), EPS Growth (35%)	30%
Valuation	Sector-Relative P/E vs. 12 sector medians; P/B penalty if ROE < 20%	15%
Financial Health	Debt-to-Equity ratio (threshold-based)	10%
FCF Quality	Free Cash Flow Yield (FCF / Market Cap)	10%

Rather than applying a universal P/E benchmark, the valuation pillar references a sector-specific dictionary of median P/E multiples across twelve industry classifications (for example, Technology at 32×, Energy at 13×, and Real Estate at 38×).

A technology stock trading at 40× earnings is therefore evaluated against sector-appropriate expectations rather than a single cross-sector threshold, reflecting the material differences in growth rates and valuation norms across industries.

For companies exhibiting financial distress (defined here as negative ROE or negative net margins), the fundamental score is capped at 35/100 regardless of other pillar outcomes. This ceiling prevents the composite score from overstating the investment quality of structurally impaired companies.

```

190     def calculate_fundamental(self, info):
191         rev_growth = info.get('revenueGrowth')
192         margins = info.get('profitMargins')
193         op_margins = info.get('operatingMargins')
194         fcf = info.get('freeCashflow')
195         market_cap = info.get('marketCap')
196         eps_growth = info.get('earningsGrowth')
197
198         insider_buys = info.get('insider_buys', 0)
199         insider_sells = info.get('insider_sells', 0)
200
201         is_distressed = (roe is not None and roe < 0) or (margins is not None and margins < 0)
202
203         sector_pe_median = {
204             'Technology': 32, 'Communication Services': 22,
205             'Consumer Cyclical': 25, 'Consumer Defensive': 22,
206             'Healthcare': 28, 'Biotechnology': 35,
207             'Financial Services': 14, 'Financials': 14,
208             'Industrials': 20, 'Basic Materials': 16,
209             'Energy': 13, 'Utilities': 18, 'Real Estate': 38,
210             'Semiconductor': 30, 'Software': 35, 'Retail': 20, 'Automotive': 14,
211         }
212         pe_median = sector_pe_median.get(self.sector, 20)

```

Figure 3.13: scorers.py: calculate_fundamental() method showing profitability pillar computation and sector P/E dictionary

3.9.5 Composite Score Calculation

The four module scores are combined using the following weighted formula:

```

1 base_composite = (score_fund * 0.40) + (score_social * 0.25)
2                 + (score_tech * 0.20) + (score_deriv * 0.15)

```

The module weights are defined in Table 3.4:

Fundamentals carry the highest weight (40%) on the basis that the financial economics literature consistently identifies profitability, growth, and valuation as the primary drivers of long-term equity returns. Sentiment receives 25% given the well-documented short-to-medium-term price impact of news, particularly in markets characterised by algorithmic trading activity. Technical signals are weighted at 20% to capture trend confirmation and market structure, while derivatives receive 15% as a proxy for institutional positioning.

Table 3.4: Composite Score Module Weights

Signal Module	Weight	Description
Fundamentals	40%	Balance sheet strength, profitability, valuation
AI Sentiment	25%	News headline classification via Groq LLM
Technical	20%	Trend, momentum, volatility indicators
Derivatives	15%	Options activity, short interest, implied volatility

Following base composite calculation, an insider buying bonus of up to +10 points is applied. The bonus is computed as the number of insider buy transactions over the preceding twelve months, multiplied by 1.5, capped at 15 points, and hard-limited to a +10 point maximum contribution to the composite. This adjustment reflects the academic literature’s treatment of insider buying as a high-conviction bullish signal.

A non-linear stretching function is then applied to increase score separation at the distribution extremes:

```

1 if composite_raw >= 72:
2     composite = 72 + (composite_raw - 72) * 1.4
3 elif composite_raw <= 32:
4     composite = 32 - (32 - composite_raw) * 1.4
5 else:
6     composite = composite_raw

```

This function amplifies the contrast between clearly bullish and clearly bearish readings, reducing score compression around the neutral midpoint.

3.10 Testing

3.10.1 Manual Testing Approach

The system was tested manually using a representative sample of stocks across sectors and market conditions, including technology mega-caps (AAPL, NVDA,

```
241 # Weights: Fundamentals 40% | Sentiment 25% | Technical 20% | Derivatives 15%
242 base_composite = (
243     score_fund * 0.40 +
244     score_social * 0.25 +
245     score_tech * 0.20 +
246     score_deriv * 0.15
247 )
248
249 insider_buys = meta_fund.get('insider_buys', 0)
250 insider_sells = meta_fund.get('insider_sells', 0)
251 insider_booster = meta_fund.get('insider_booster', 0)
252 composite = min(100, base_composite + insider_booster)
253
```

Figure 3.14: `main.py`: composite score calculation block showing weighting, insider booster, and stretching logic

MSFT), energy stocks (XOM), financially distressed companies with negative ROE, and small-cap stocks with limited options chains. Each test case verified that ticker resolution was correct, all four module scores fell within the valid 0–100 range, the distressed company cap was applied where applicable, sentiment classifications and source attributions were accurate, and the candlestick chart rendered with all three indicator overlays present.

3.10.2 Expected Dashboard Output

The figures below show the dashboard in its fully rendered state: data retrieved, all four module scores calculated, and the composite output displayed. This represents the expected interface under normal operating conditions.

3.10.3 Edge Case Handling

The following edge cases were identified and validated during development.

3.10.3.1 Tickers with Fewer Than 50 Days of Price History

When a recently listed stock lacks sufficient price history for technical indicator computation, `calculate_technical()` returns `(0, {})`, and the Technical Analysis panel renders at zero. The remaining three modules are unaffected. The figure below shows this behaviour using MAIR, a recently listed ticker with limited market history.

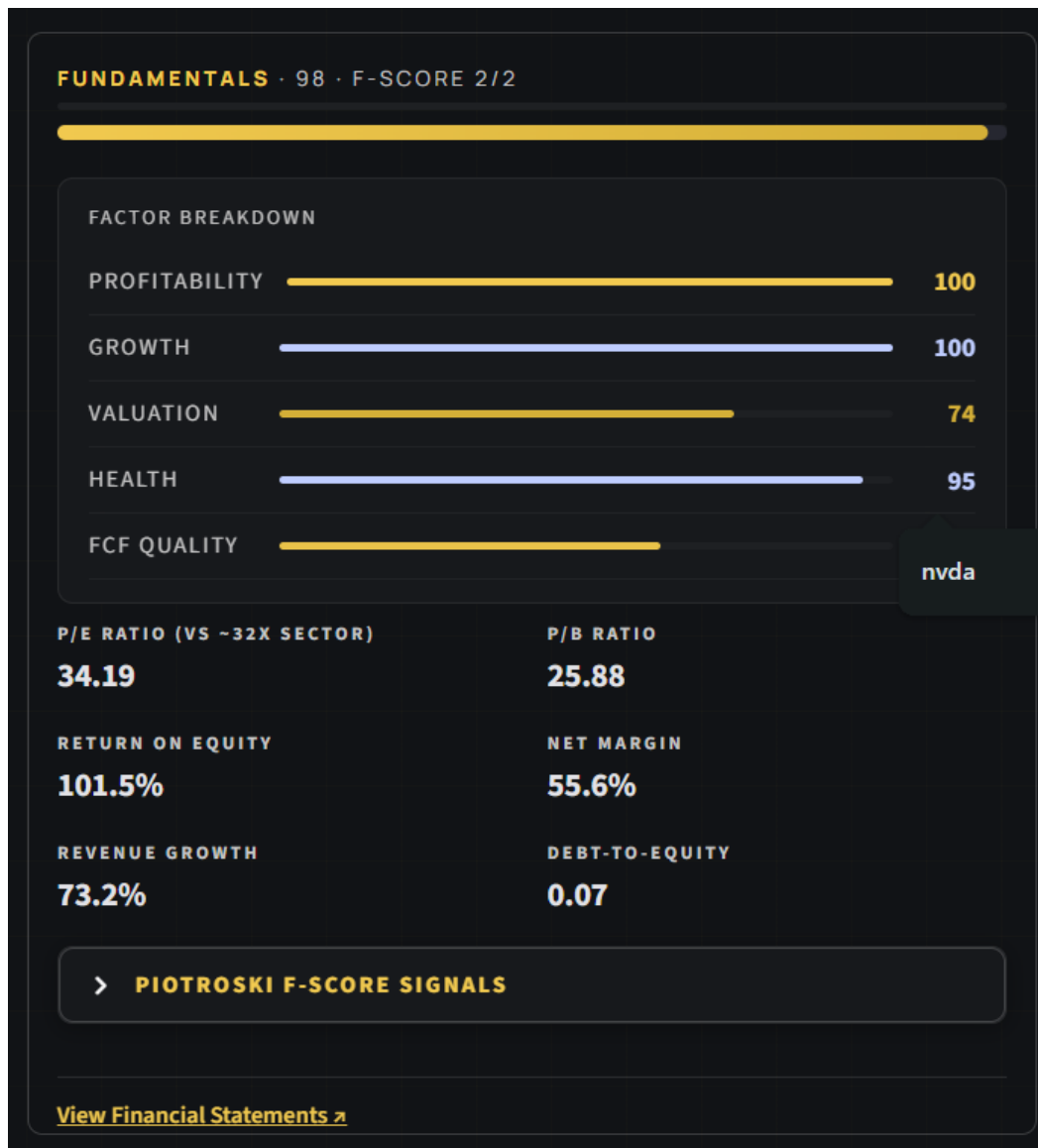


Figure 3.15: Dashboard Output Panel 1

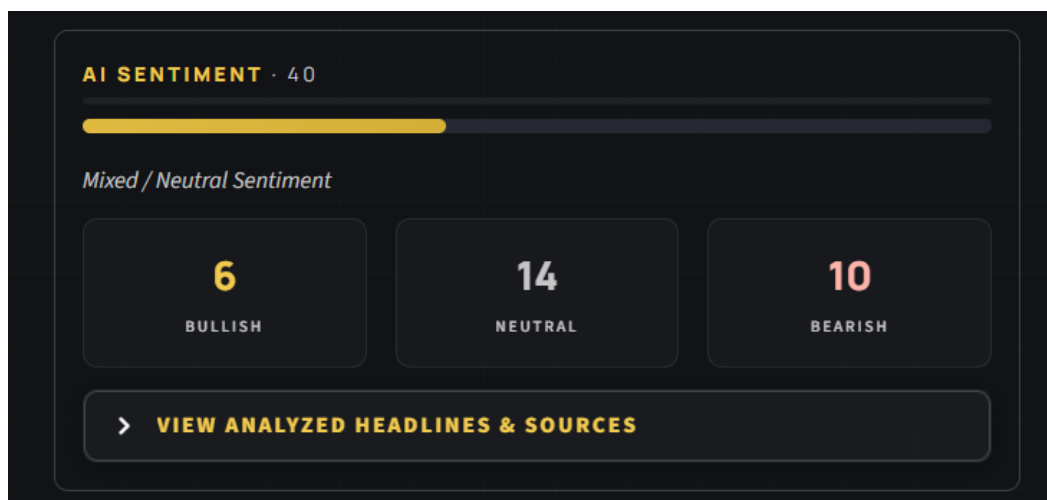


Figure 3.16: Dashboard Output Panel 2

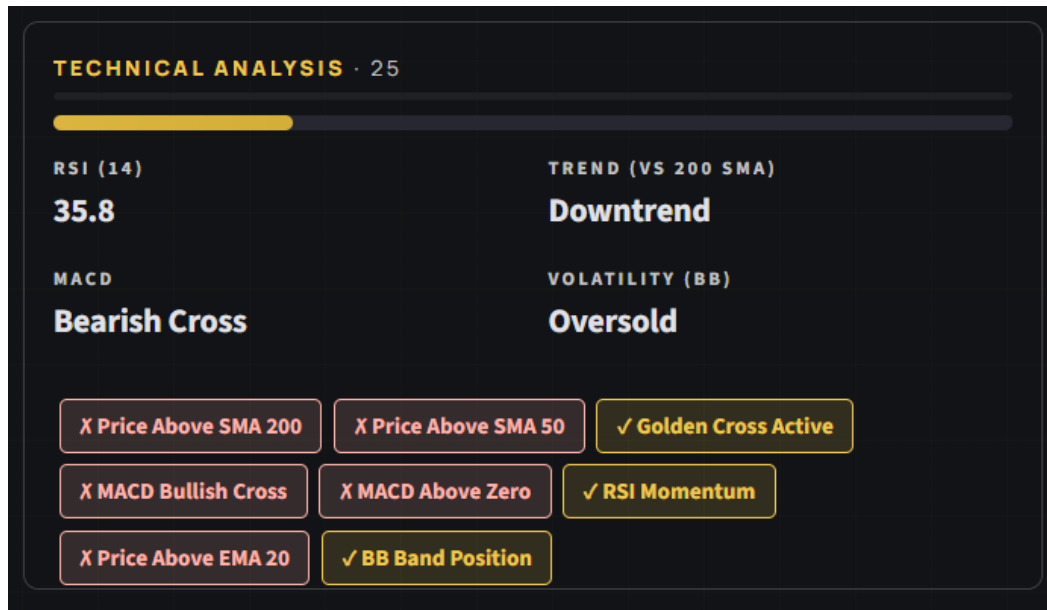


Figure 3.17: Dashboard Output Panel 3

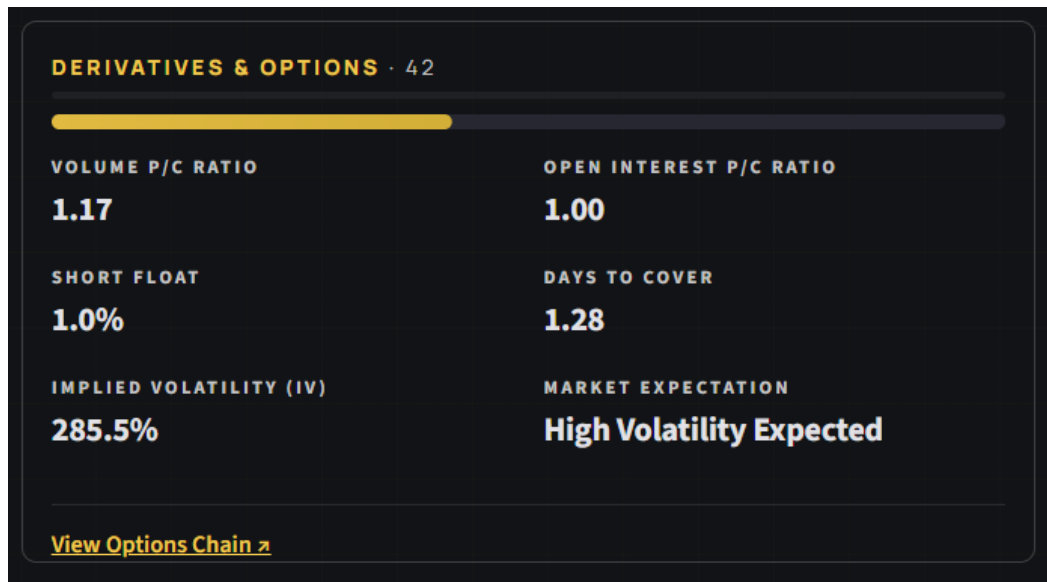


Figure 3.18: Dashboard Output Panel 4

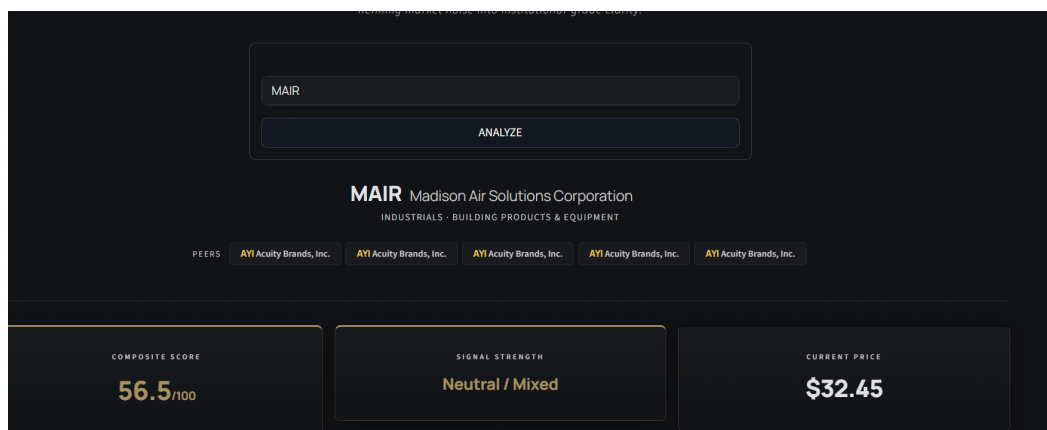


Figure 3.19: Edge Case: MAIR overview with limited price history

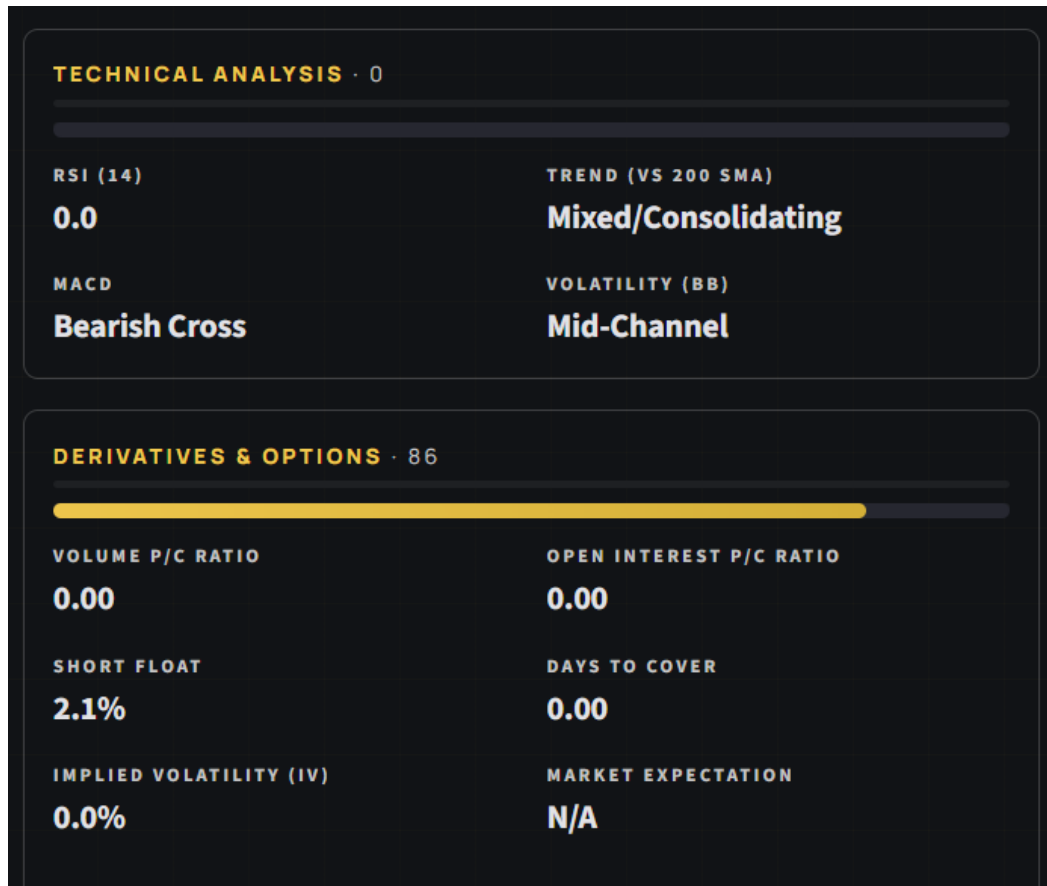


Figure 3.20: Edge Case: Technical panel rendered at zero for insufficient history

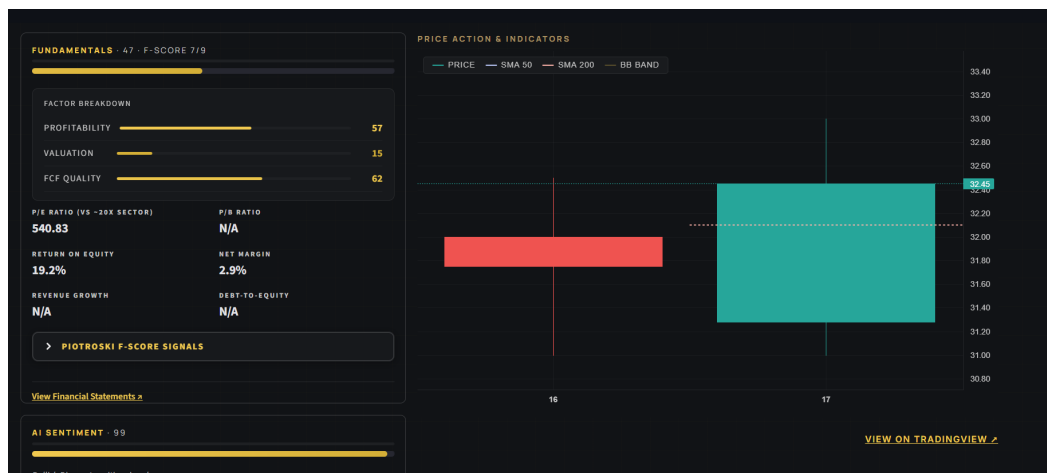


Figure 3.21: Edge Case: Fundamental Panel and Chart for MAIR

3.10.3.2 Stocks with No Listed Options

When no valid options chain exists for a given ticker, `calculate_derivative()` receives `{'valid': False}` and returns `(0, {})`. Recently listed stocks may simultaneously lack sufficient price history for the technical module, in which case both

the Technical Analysis and Derivatives & Options panels render at zero. This is expected behaviour: both modules are responding correctly to missing data, not failing. The fundamental and sentiment modules continue to produce scores independently. The ARXS example below illustrates this combined fallback behaviour.

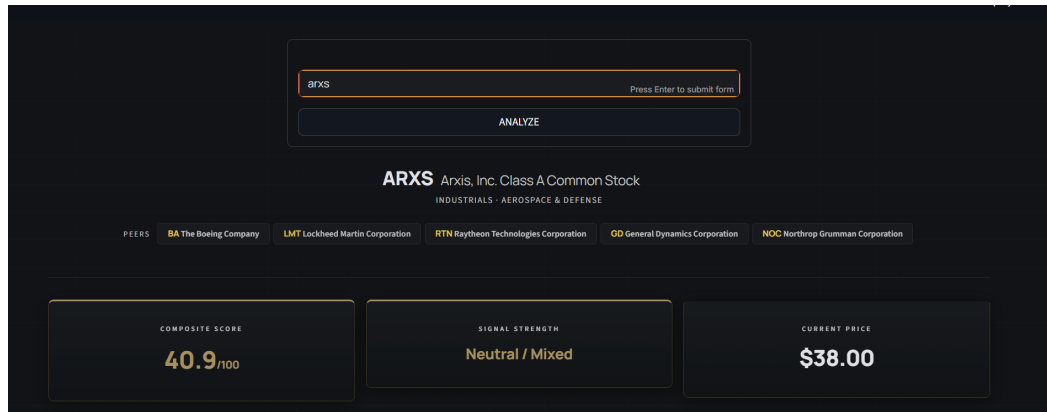


Figure 3.22: Edge Case: ARXS overview for a recently listed stock without options data

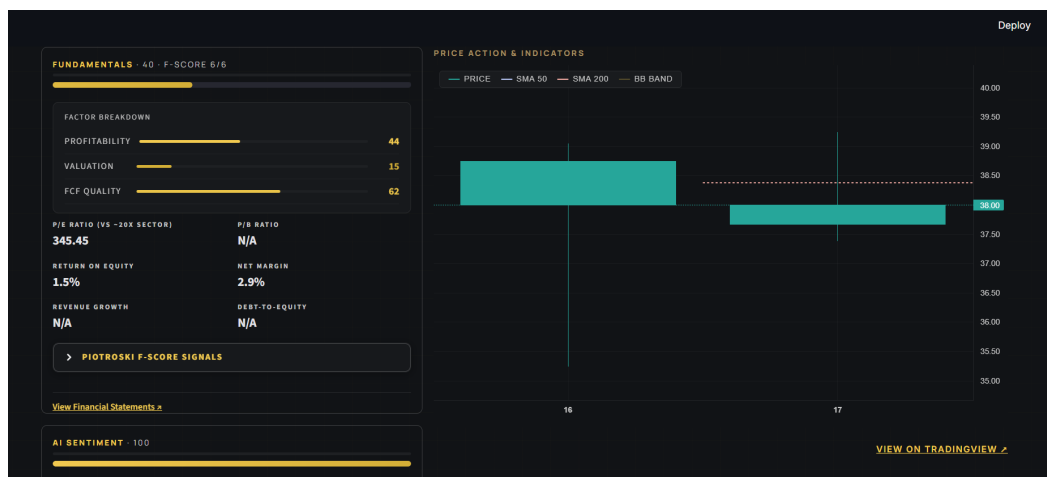


Figure 3.23: Edge Case: ARXS dashboard panels with fundamentals and chart

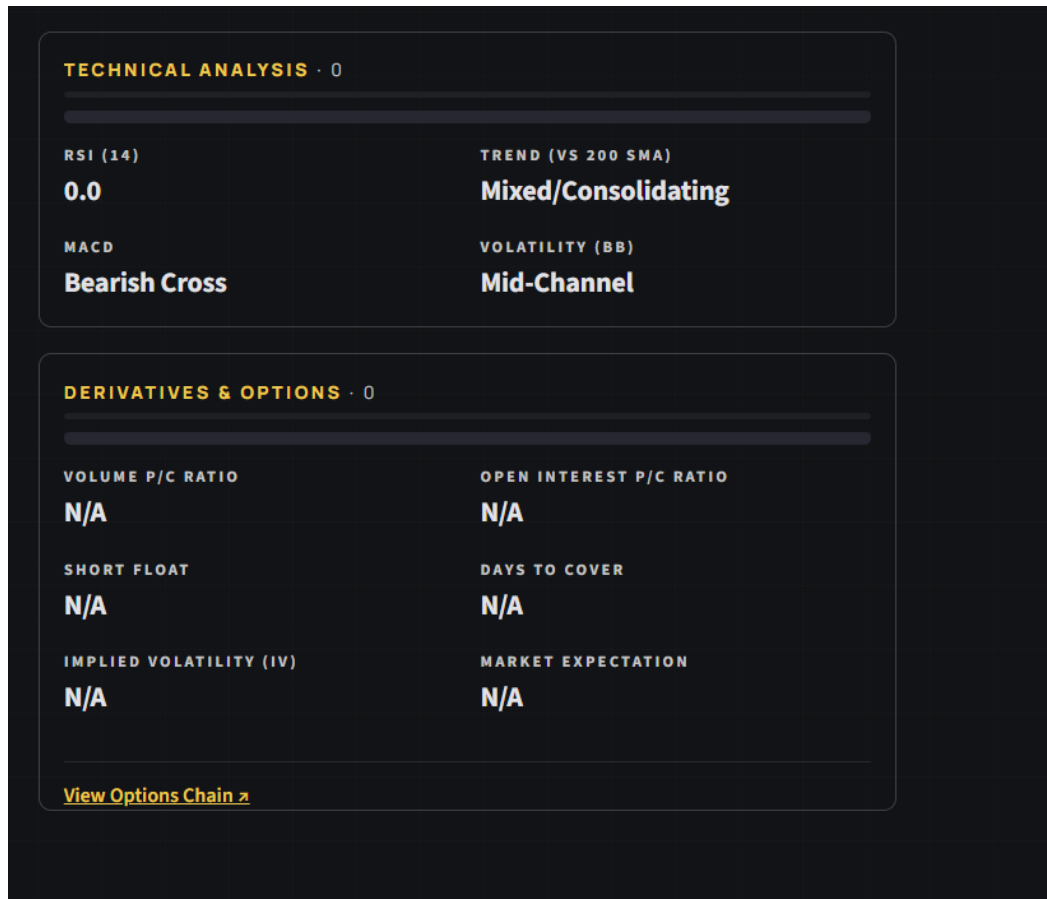


Figure 3.24: Edge Case: ARXS technical and derivatives panels correctly rendered at zero

3.10.3.3 FinViz Rate-Limiting or Empty Headline Retrieval

If `_scrape_finviz()` returns an empty list because of rate limiting or request failure, `calculate_social()` returns `(0, {'error': '...'})`. The sentiment panel then renders an error state rather than interrupting the full dashboard.

3.10.3.4 Groq API Quota Exhaustion Across All Keys

If all available Groq API keys are exhausted, the key-rotation loop terminates and `get_social_sentiment()` returns `{'error': 'Groq AI Services are busy...'}`. This makes the failure explicit while preserving graceful degradation in the UI.

3.10.4 Unit Testing

Pytest is used to implement modular test cases across the codebase. Tests are organised into separate files by concern, each targeting a specific module: data load-

ing, scoring logic, and utility functions. Shared test state is managed through a centralised fixture file.

3.10.4.1 Test Structure Overview

3.10.4.2 Shared Test Fixtures (conftest.py)

The `conftest.py` file defines reusable pytest fixtures that are automatically injected into test functions via parameter declaration. This eliminates boilerplate setup code and keeps individual tests focused on their assertions.

`tests/conftest.py`:

```
1  import pytest
2  import pandas as pd
3  import numpy as np
4
5  @pytest.fixture
6  def mock_stock_data():
7      """Creates 100 days of fake stock price data for technical analysis testing."""
8      dates = pd.date_range(start="2023-01-01", periods=100)
9      data = {
10         'Open': np.linspace(100, 150, 100),
11         'High': np.linspace(105, 155, 100),
12         'Low': np.linspace(95, 145, 100),
13         'Close': np.linspace(102, 152, 100),
14         'Volume': np.random.randint(1000, 5000, 100)
15     }
16     return pd.DataFrame(data, index=dates)
17
18 @pytest.fixture
19 def mock_financials():
20     """Creates a mock Series for fundamental scoring."""
21     return pd.Series([100, 80, 60], index=[0, 1, 2]) # Current, Previous, Year-Before
```

Figure 3.25: Testing: `conftest.py`

3.10.4.3 Data Loader Tests (test_data_loader.py)

This file validates the behaviour of the `DataLoader` class, with particular focus on API key rotation under rate-limit conditions. Given that the system maintains a pool of keys to handle Groq API quota exhaustion, correct rotation behaviour is a critical correctness requirement.

`tests/test_data_loader.py`:

3.10.4.4 Scoring Engine Tests (test_scorers.py)

This file covers the `ScoringEngine` class across three scenarios: normal operation with valid input data, graceful degradation with empty input, and the Piotroski F-Score calculation logic.

```
tests > test_data_loader.py > ...
1 import pytest
2 from unittest.mock import patch, MagicMock
3 from data_loader import DataLoader
4
5 @patch('data_loader.Groq')
6 def test_groq_key_rotation(mock_groq_class):
7     # Setup mock to simulate a failure on the first key and success on the second
8     instance = mock_groq_class.return_value
9     instance.chat.completions.create.side_effect = [Exception("Rate Limit"), MagicMock()]
10
11     loader = DataLoader()
12     # Force some mock keys into the pool
13     loader.api_key_pool = ["key1", "key2"]
14
15     # This calls the method that uses Groq
16     loader.get_competitors("AAPL", "Apple", "Tech", "iPhone")
17
18     # Verify it tried to call Groq
19     assert mock_groq_class.call_count >= 1
```

Figure 3.26: Testing: test_data_loader.py

tests/test_scorers.py:

```

tests > test_scorers.py > test_technical_invalid_data
1  import pytest
2  import pandas as pd
3  from scorers import ScoringEngine
4
5  def test_technical_scoring_with_data(mock_stock_data):
6      engine = ScoringEngine()
7      score, signals = engine.calculate_technical(mock_stock_data)
8
9      assert 0 <= score <= 100
10     assert isinstance(signals, dict)
11     # Check that signals were actually generated
12     assert len(signals) > 0
13
14 def test_technical_invalid_data():
15     engine = ScoringEngine()
16     # Create an empty dataframe correctly
17     empty_df = pd.DataFrame()
18     score, signals = engine.calculate_technical(empty_df)
19
20     # Based on scorers.py, empty DF should return 0
21     assert score == 0
22     assert signals == {}
23
24 def test_piotroski_f_score_logic():
25     engine = ScoringEngine()
26     # The engine expects the dataframes to be passed differently.
27     # We will pass an empty DataFrame as the primary data source.
28     score, signals = engine.calculate_fundamental(
29         pd.DataFrame()
30     )
31     assert isinstance(score, (int, float))
32     assert isinstance(signals, dict)

```

Figure 3.27: Testing: test_scorers.py

3.10.4.5 Utility Function Tests (test_utils.py)

This file tests two standalone functions exported from `utils.py`: `get_rating()`, which maps a numeric score to a label and colour string, and `normalize()`, which scales an input value to the 0–100 range.

tests/test_utils.py

```

tests > test_utils.py > ...
1  from utils import get_rating, normalize
2
3  def test_get_rating_bounds():
4      # Test Strong Bullish
5      label, color = get_rating(85)
6      assert label == "Strong Bullish"
7      assert color == "#f2ca50"
8
9      # Test Strong Bearish
10     label, color = get_rating(10)
11     assert label == "Strong Bearish"
12     assert color == "#ff4444"
13
14 def test_normalize_function():
15     # Test standard normalization
16     assert normalize(5, 0, 10) == 50.0
17     # Test capping at 100
18     assert normalize(15, 0, 10) == 100
19     # Test bottoming at 0
20     assert normalize(-5, 0, 10) == 0

```

Figure 3.28: Testing: test_utils.py

3.10.4.6 How to Run the Tests

```

PS C:\Users\umers\OneDrive\Desktop\thesis 2\project> python -m pytest tests/
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\umers\OneDrive\Desktop\thesis 2\project
plugins: anyio-4.12.1, langsmith-0.7.7
collected 6 items

tests\test_data_loader.py . [ 16%]
tests\test_scorers.py ... [ 66%]
tests\test_utils.py .. [100%]

===== 6 passed in 1.65s =====
PS C:\Users\umers\OneDrive\Desktop\thesis 2\project>

```

Figure 3.29: Testing: test results

```

1 # Run all tests
2 pytest tests/ or python -m pytest tests/

```



```
3 # Run a specific test file
4 pytest tests/test_utils.py
```

3.10.4.7 Summary of Test Cases

All test cases are deterministic and isolated from live market data or active API connections. Mocking and fixture-based data keep them fast, repeatable, and compatible with continuous integration pipelines.

Chapter 4

Conclusion and Future Work

4.1 Introduction

The system described in this thesis demonstrates that multisignal stock evaluation is achievable outside institutional infrastructure — no Bloomberg terminal, no proprietary data feeds — using only open-source libraries and low-cost inference APIs. Whether the design decisions behind it hold up is a harder question. Rule-based scoring is transparent by construction, but transparency does not guarantee that the weights are right, that the thresholds generalise, or that the composite score means the same thing across different market conditions. This chapter addresses what the project achieved, where those limits became visible during development, and what a more complete version of the system would require.

4.2 Conclusion

This thesis designed and built an AI-powered stock evaluation system that combines four signal types (technical price analysis, LLM-based news sentiment classification, equity derivatives data, and multi-pillar fundamental analysis) into a single composite score.

The Groq-hosted LLM inference is worth noting specifically: classifying thirty news headlines in under two seconds makes sentiment analysis practical inside an interactive dashboard, which was not feasible with the slower APIs available previously.

The five-pillar fundamental scoring model improves on single-metric approaches like P/E ratio alone by incorporating sector-relative benchmarking and a distressed company score cap. The trend-aware derivatives module addresses a common limitation of mechanical signal processing by interpreting the same options or short-selling data differently depending on the prevailing price trend.

The Streamlit dashboard presents all computed signals, sub-scores, and raw metrics in a single interface that does not require financial expertise to navigate. The expandable Piotroski badge and headline viewer serve users who want to investigate further; the top-level metric cards and signal strength rating give a fast summary for everyone else.

The prototype supports the core premise: combining multiple signal types produces a richer and more grounded stock evaluation than any single method, and a system of this kind is achievable within the resource constraints of a university thesis project.

4.3 Future Work

Several extensions would improve the system’s scope and practical usefulness.

- **Multi-ticker screening:** Accepting a comma-separated list of tickers and returning a ranked table would let users screen a full watchlist in one query rather than one stock at a time.
- **Empirical weight optimisation:** The module weights (40/25/20/15) were derived from academic literature. A backtesting study over a broad sample of US stocks could determine whether alternative weights produce better return predictions.
- **Additional sentiment sources:** Extending the pipeline beyond FinViz-aggregated news to include Reddit, Twitter/X, and Stocktwits would capture retail investor sentiment, which tends to diverge from institutional news flow.
- **International markets:** Supporting non-US exchanges (LSE, Euronext, TSE, HKEX) would require adjusting sector P/E benchmarks for local norms and sourcing appropriate news data.

- **SHAP-based attribution:** Applying SHAP analysis to the composite score would show how much each sub-signal contributed to a given result, making the model more interpretable.
- **Market hours detection:** The system should label price data from closed sessions clearly, so users do not mistake stale prices for live quotes.

Acknowledgements

I am grateful to my supervisor, Md. Easin Arafat, for his guidance throughout this project. His feedback on the system architecture and technical decisions was consistently constructive, and he engaged seriously with the problems I brought to him.

I would also like to thank the Faculty of Informatics at Eötvös Loránd University for the academic environment that made this work possible.

I wish to acknowledge the open-source community — particularly the maintainers of yfinance, ta, Streamlit, and the Groq SDK — whose freely available tools made it realistic to complete this as a single-author project.

Above all, I thank my father, Muhammad Aslam. His belief in education and in the value of hard work shaped everything that led to this degree. Completing it abroad would not have been possible without the foundation he built.

Finally, I thank my family and friends for their support throughout my studies.

Bibliography

- [1] Yahoo Finance / yfinance. (2024). yfinance: Download market data from Yahoo! Finance's API. Available at: <https://github.com/ranaroussi/yfinance>
- [2] Bukosza, D. (2021). ta: Technical Analysis Library for Python. Available at: <https://technical-analysis-library-in-python.readthedocs.io/>
- [3] Groq, Inc. (2024). Groq API Documentation. Available at: <https://console.groq.com/docs>
- [4] Meta AI. (2024). Llama 3: Open Foundation and Fine-Tuned Chat Models. Available at: <https://ai.meta.com/llama/>
- [5] Streamlit Inc. (2024). Streamlit Documentation. Available at: <https://docs.streamlit.io/>
- [6] Piotroski, J.D. (2000). Value Investing: The Use of Historical Financial Statement Information to Separate Winners from Losers. *Journal of Accounting Research*, 38, 1-41.
- [7] Plotly Technologies Inc. (2024). Plotly Python Open Source Graphing Library. Available at: <https://plotly.com/python/>
- [8] BeautifulSoup / Richardson, L. (2023). BeautifulSoup Documentation. Available at: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- [9] FinViz LLC. (2024). FinViz - Financial Visualizations. Available at: <https://finviz.com/>
- [10] Fama, E.F. & French, K.R. (1993). Common Risk Factors in the Returns on Stocks and Bonds. *Journal of Financial Economics*, 33(1), 3-56.

- [11] Tetlock, P.C. (2007). Giving Content to Investor Sentiment: The Role of Media in the Stock Market. *The Journal of Finance*, 62(3), 1139-1168.
- [12] Python Software Foundation. (2024). Python Programming Language. Available at: <https://www.python.org/>

List of Figures

2.1	Project File Structure in VS Code	14
2.2	Main Dashboard - Ticker Search Input	16
2.3	Main Dashboard - Landing Page Overview	16
2.4	Main Dashboard - Workflow Explanation	17
2.5	Main Dashboard - Fetching Screen During Analysis	17
2.6	Top Metric Cards - Composite Score, Signal Strength, and Current Price	18
2.7	Insider Activity Banner - showing buy/sell counts.	19
2.8	Fundamentals Panel	20
2.9	AI Sentiment Panel	21
2.10	Technical Panel	22
2.11	Derivatives Panel	22
2.12	Interactive Candlestick Chart	23
3.1	System Architecture Diagram	25
3.2	Class Diagram	26
3.3	Use Case Diagram	27
3.4	User Sequence Diagram	28
3.5	<code>data_loader.py</code> — <code>DataLoader</code> class with <code>get_technical_data()</code> , <code>get_fundamental_data()</code> , <code>get_derivative_data()</code> , <code>get_social_sentiment()</code> methods	30
3.6	<code>data_loader.py</code> — <code>convert_name_to_ticker()</code> function showing Yahoo Finance search API call with US exchange filtering	31
3.7	<code>scorers.py</code> : <code>ScoringEngine</code> class header and <code>calculate_technical()</code> method showing signal weight definitions	31
3.8	<code>utils.py</code> : <code>normalize()</code> and <code>get_rating()</code> functions	32
3.9	<code>main.py</code> : composite score weighting formula and score stretching logic	33

3.10	<code>scorers.py</code> : <code>calculate_technical()</code> method body with signal list and weighted scoring computation	34
3.11	<code>scorers.py</code> : <code>calculate_social()</code> method showing sentiment accumula- tion and power-weighted scoring formula	35
3.12	<code>scorers.py</code> : <code>calculate_derivative()</code> method showing trend- aware short float scoring branch	36
3.13	<code>scorers.py</code> : <code>calculate_fundamental()</code> method showing profitability pil- lar computation and sector P/E dictionary	37
3.14	<code>main.py</code> : composite score calculation block showing weighting, insider booster, and stretching logic	39
3.15	Dashboard Output Panel 1	40
3.16	Dashboard Output Panel 2	40
3.17	Dashboard Output Panel 3	41
3.18	Dashboard Output Panel 4	41
3.19	Edge Case: MAIR overview with limited price history	41
3.20	Edge Case: Technical panel rendered at zero for insufficient history	42
3.21	Edge Case: Fundamental Panel and Chart for MAIR	42
3.22	Edge Case: ARXS overview for a recently listed stock without options data	43
3.23	Edge Case: ARXS dashboard panels with fundamentals and chart	43
3.24	Edge Case: ARXS technical and derivatives panels correctly rendered at zero	44
3.25	Testing: <code>conftest.py</code>	45
3.26	Testing: <code>test_data_loader.py</code>	46
3.27	Testing: <code>test_scorers.py</code>	47
3.28	Testing: <code>test_utils.py</code>	48
3.29	Testing: test results	48

List of Tables

2.1	Hardware and Software Requirements	13
2.2	Composite Score Rating Scale	18
3.1	Technical Scoring Signals and Weights	33
3.2	Derivatives Scoring Sub-Scores and Weights	35
3.3	Fundamental Scoring Five-Pillar Model	36
3.4	Composite Score Module Weights	38